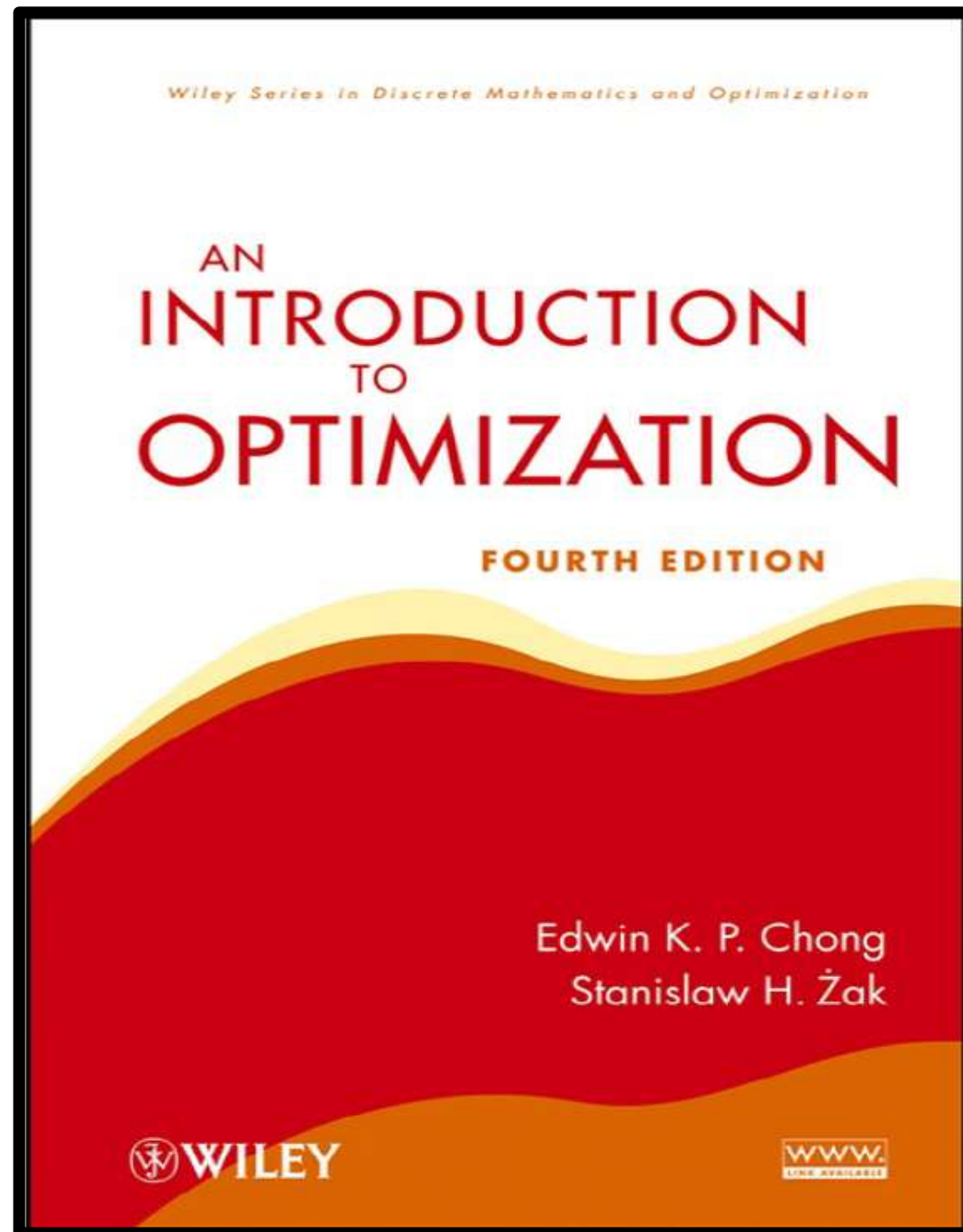


Optimization Methods

1. Introduction.
2. Greedy algorithms for combinatorial optimization.
3. LS and neighborhood structures for combinatorial optimization.
4. Variable neighborhood search, neighborhood descent, SA, TS, EC.
5. Branch and bound algorithms, and subset selection algorithms.
6. Linear programming problem formulations and applications.
7. Linear programming algorithms.
8. Integer linear programming algorithms.
9. Unconstrained nonlinear optimization and gradient descent.
10. Newton's methods and Levenberg-Marquardt modification.
11. Quasi-Newton methods and conjugate direction methods.
12. Nonlinear optimization with equality constraints.
13. Nonlinear optimization with inequality constraints.
14. Problem formulation and concepts in multi-objective optimization.
15. Search for single final solution in multi-objective optimization.
- 16: Search for multiple solutions in multi-objective optimization.

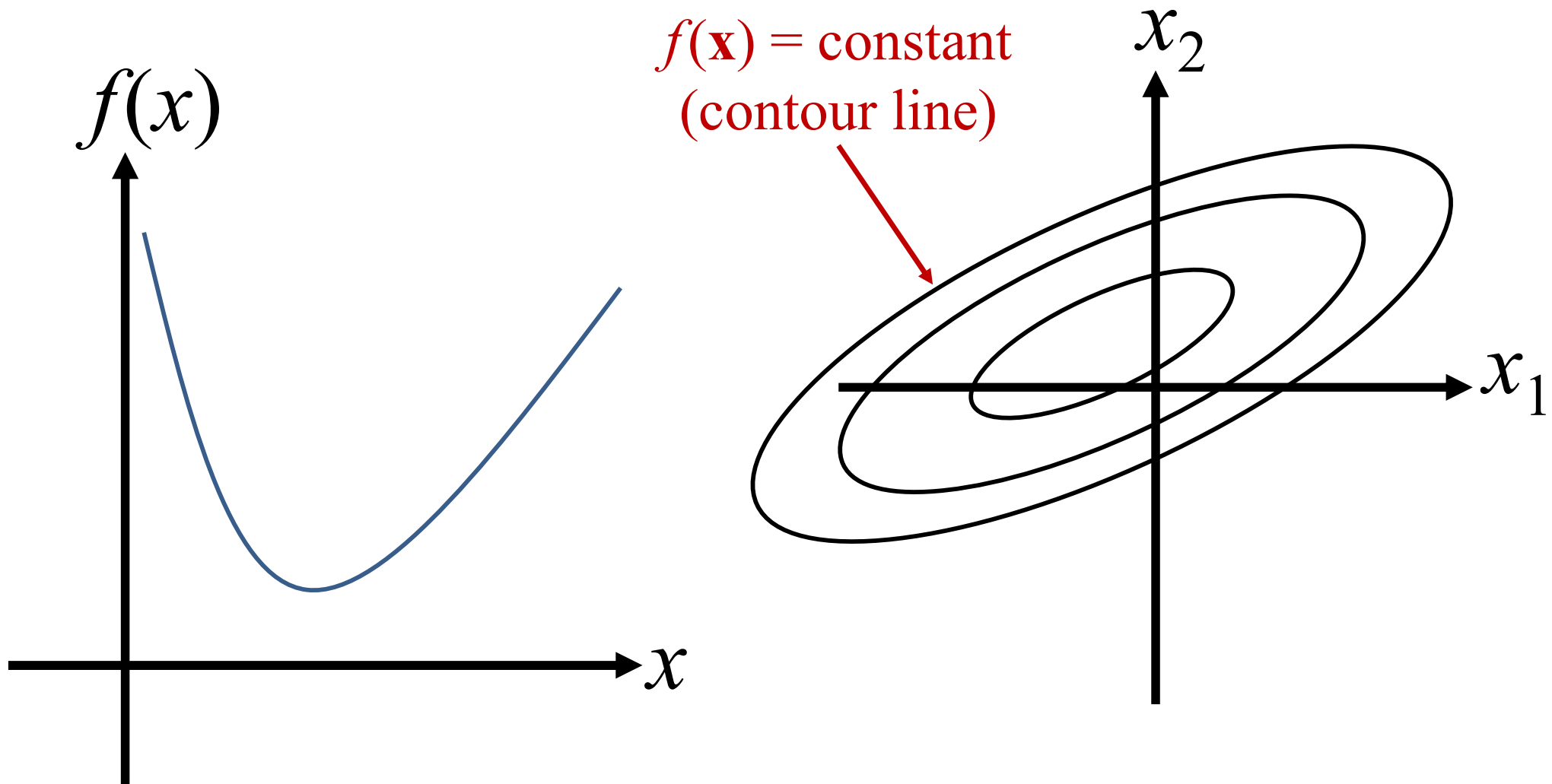
Non-Linear Optimization is based on this book.



Unconstrained nonlinear optimization Problem

Minimize $f(\mathbf{x})$, $\mathbf{x} = (x_1, x_2, \dots, x_n)^T \in \mathbb{R}^n$

$f(\mathbf{x})$: Nonlinear function

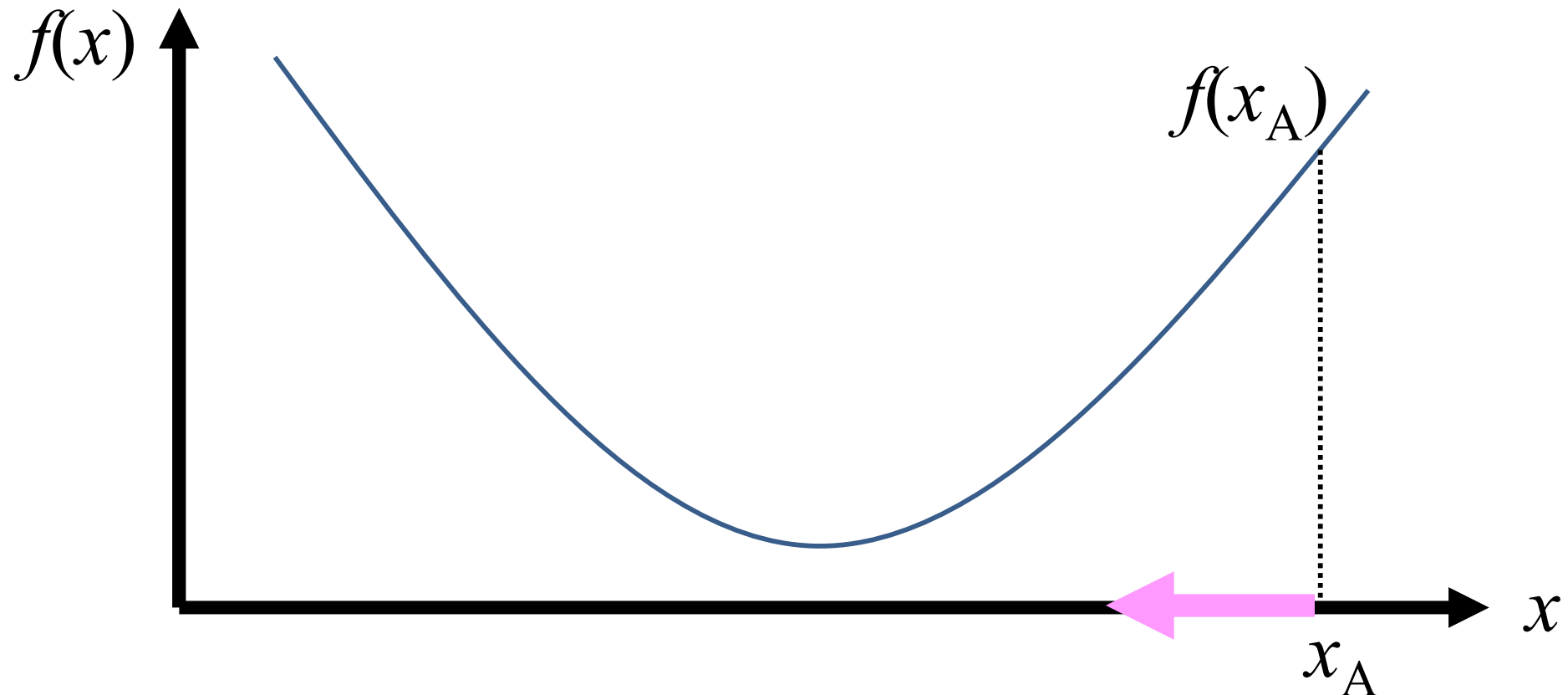


Gradient Descent (Explanation)

Nonlinear minimization problem: Minimize $z = f(x)$.

Available information: **derivative (numerical value)** df/dx

If the current solution is x_A , $df/dx > 0$. This means that $f(x)$ is increased by increasing x . Thus, we need to decrease x in order to minimize $f(x)$

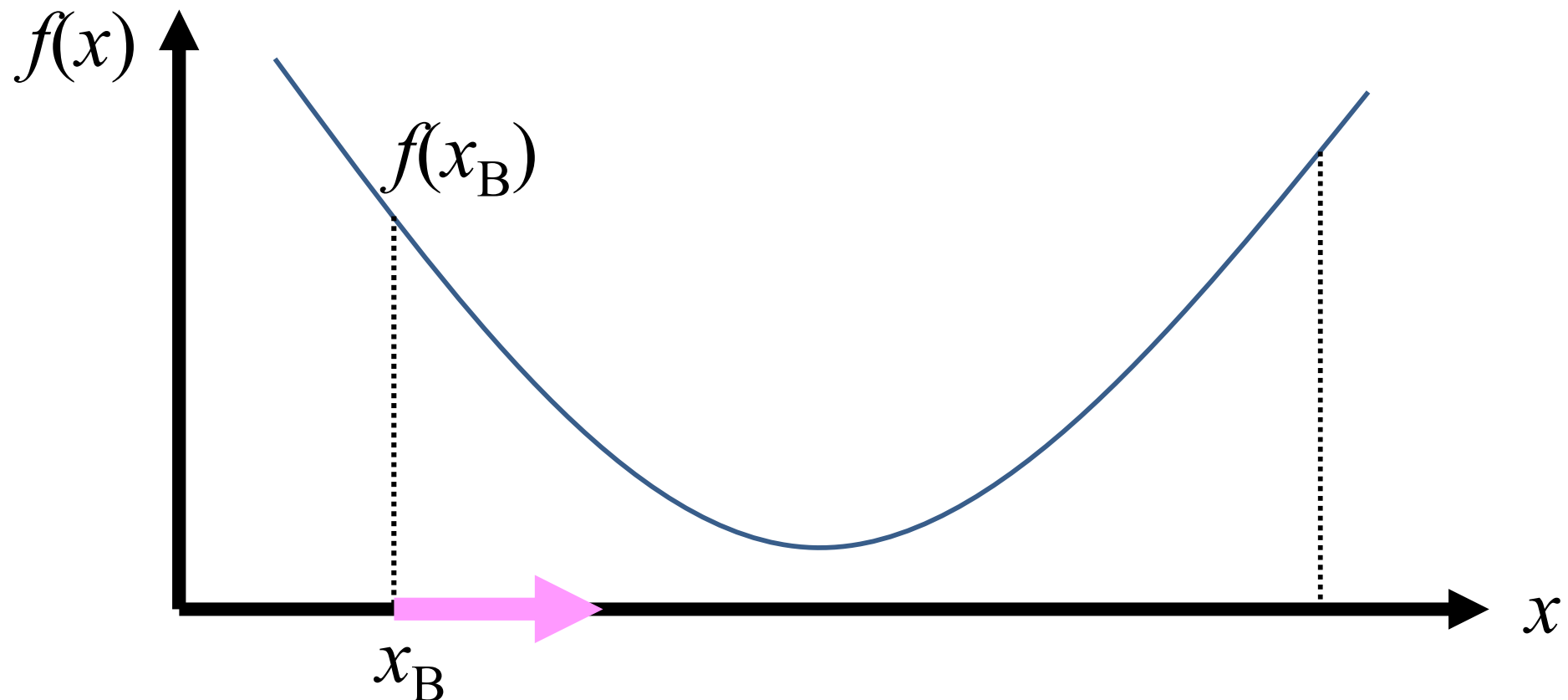


Gradient Descent (Explanation)

Nonlinear minimization problem: Minimize $z = f(x)$.

Available information: **derivative (numerical value)** df/dx

If the current solution is x_B , $df/dx < 0$. This means that $f(x)$ is decreased by increasing x . Thus, we need to increase x in order to minimize $f(x)$.



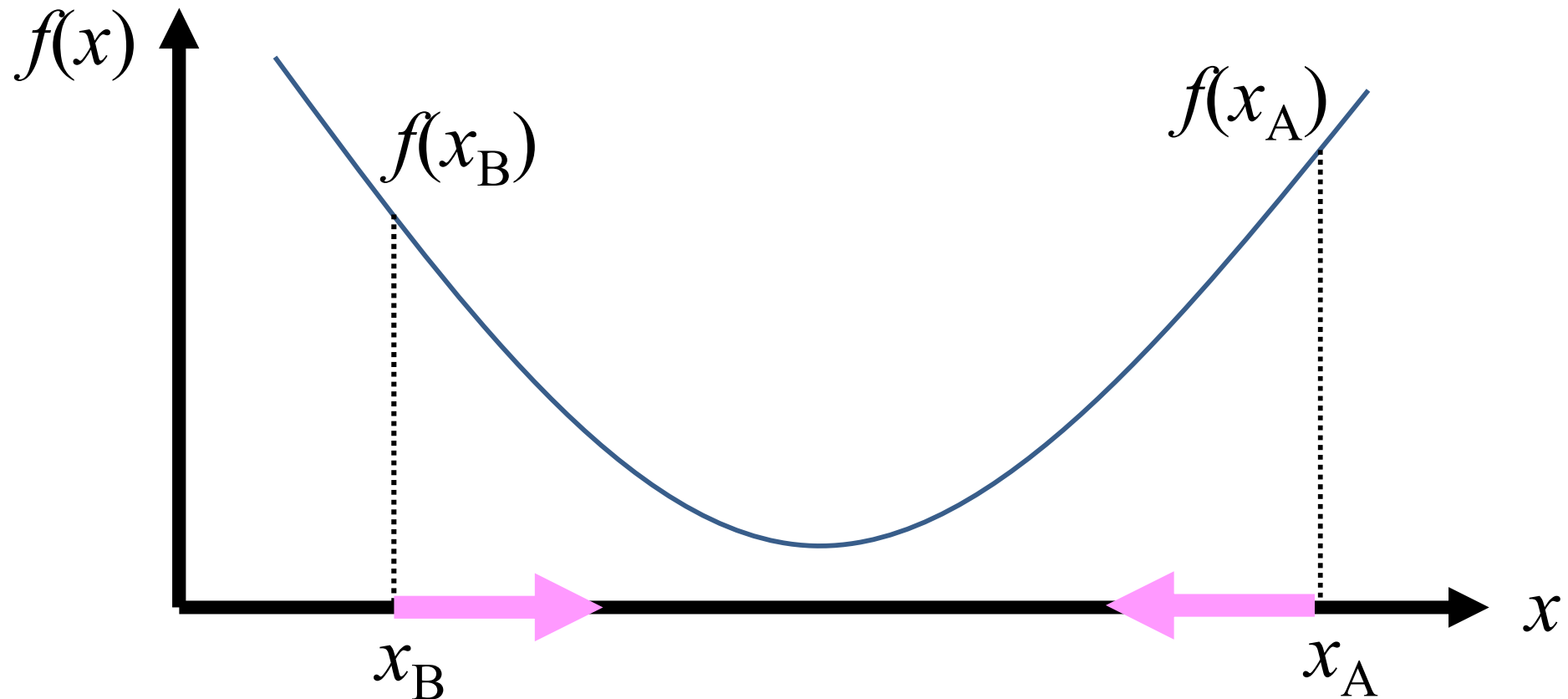
Gradient Descent (Algorithm)

$$x^{(k+1)} = x^{(k)} - \alpha^{(k)} \frac{df}{dx}$$

$x^{(1)}$: Initial value of x (i.e., initial solution)

$x^{(k)}$: Value of x at the k th iteration.

$\alpha^{(k)}$: Step size at the k th iteration (positive value).



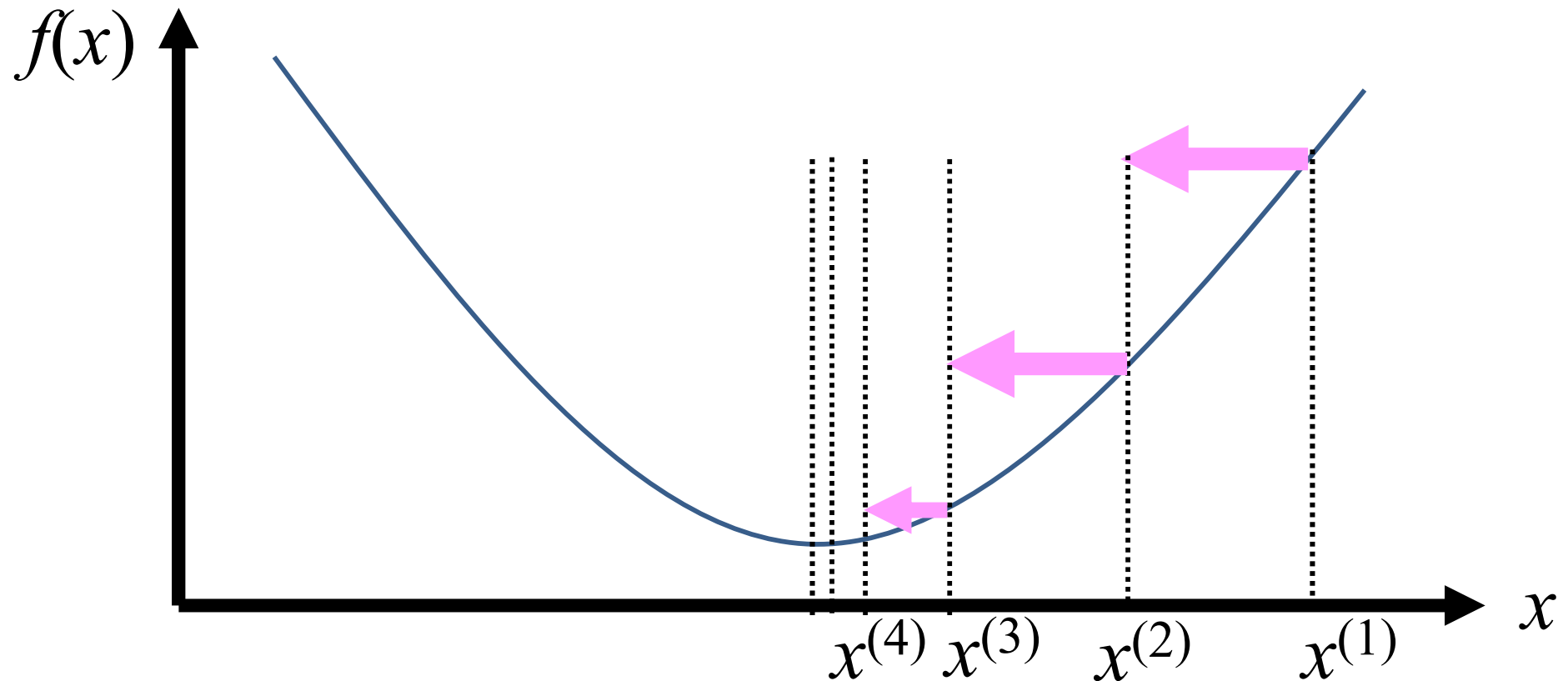
Gradient Descent (Algorithm: Constant Step Size)

$$x^{(k+1)} = x^{(k)} - \alpha \frac{df}{dx}$$

$x^{(1)}$: Initial value of x (i.e., initial solution)

$x^{(k)}$: Value of x at the k th iteration.

α : Step size (positive constant).



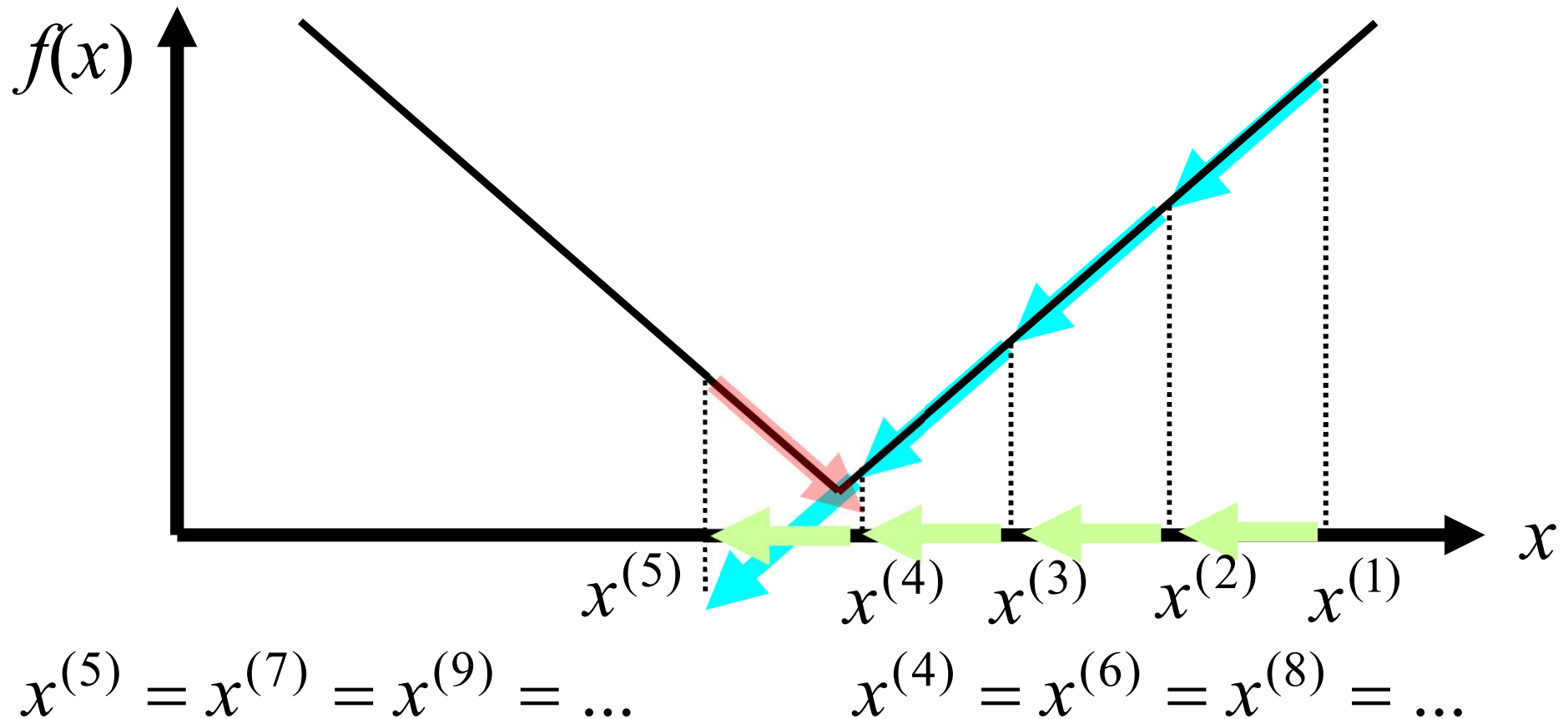
Gradient Descent (Algorithm: Constant Step Size)

$$x^{(k+1)} = x^{(k)} - \alpha \frac{df}{dx}$$

$x^{(1)}$: Initial value of x (i.e., initial solution)

$x^{(k)}$: Value of x at the k th iteration.

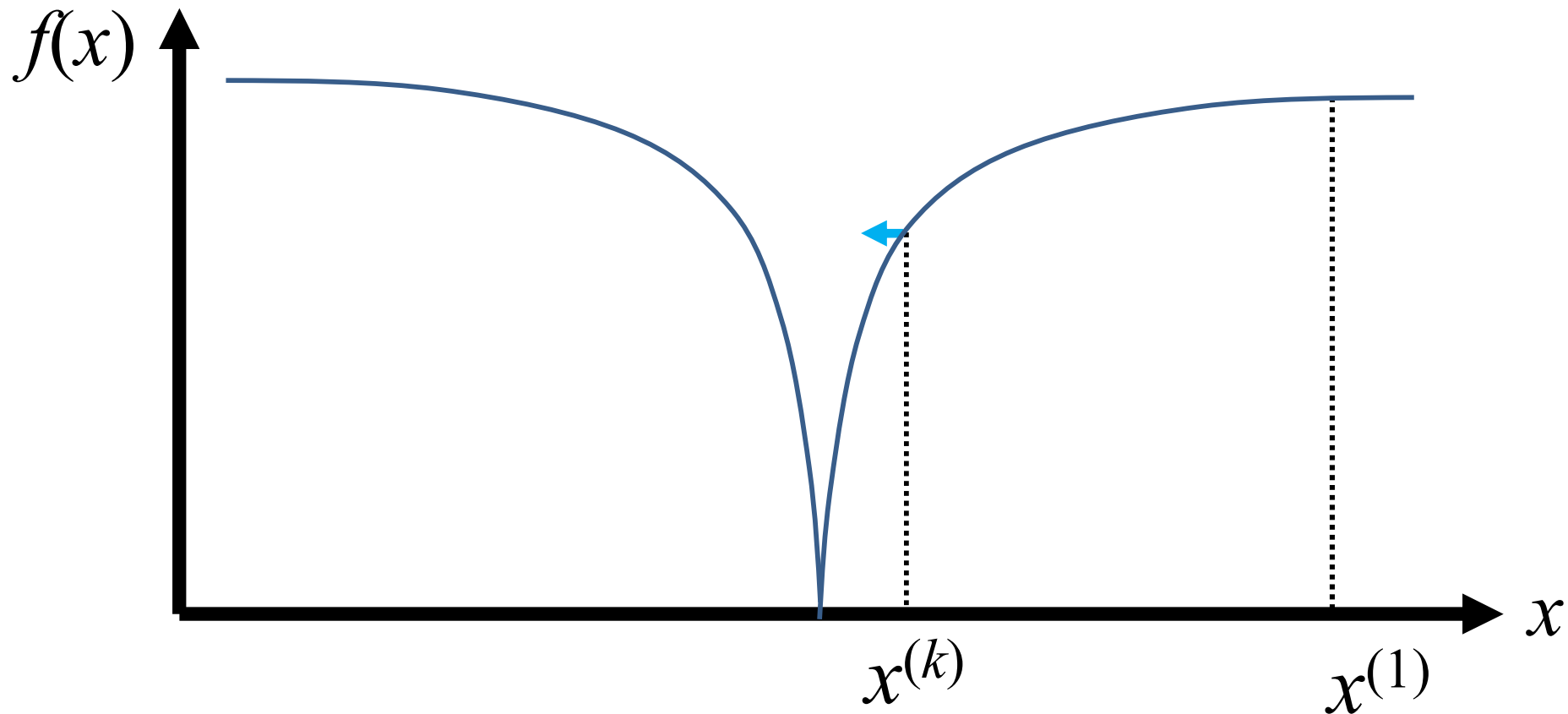
α : Step size (positive constant).



Gradient Descent (Algorithm: Constant Step Size)

$$x^{(k+1)} = x^{(k)} - \alpha \frac{df}{dx}$$

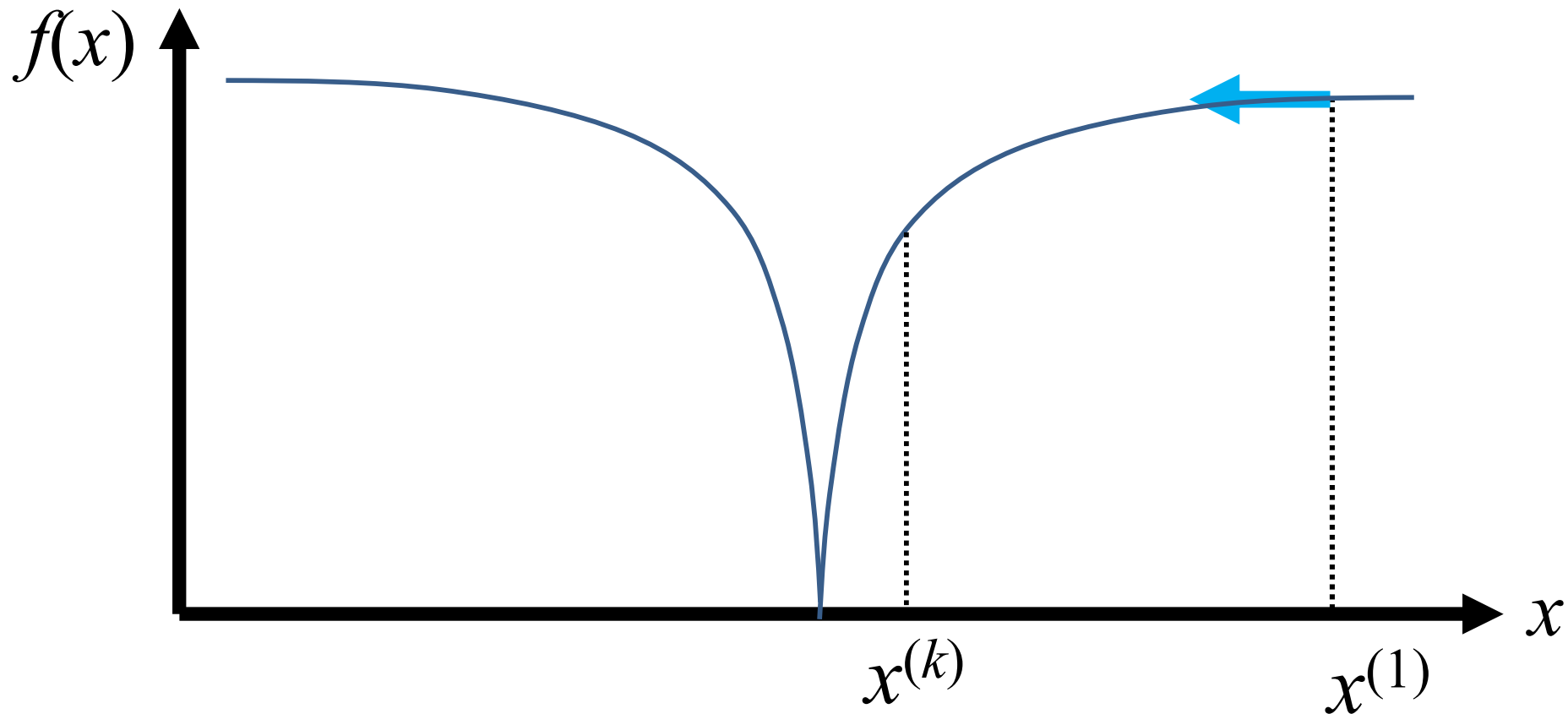
Explain a sequence of moves of a solution starting from $x^{(1)}$ when α is very small (which is appropriate for $x^{(k)}$): _____.



Gradient Descent (Algorithm: Constant Step Size)

$$x^{(k+1)} = x^{(k)} - \alpha \frac{df}{dx}$$

Explain a sequence of moves of a solution starting from $x^{(1)}$ when α is large (which is appropriate for $x^{(1)}$): _____.

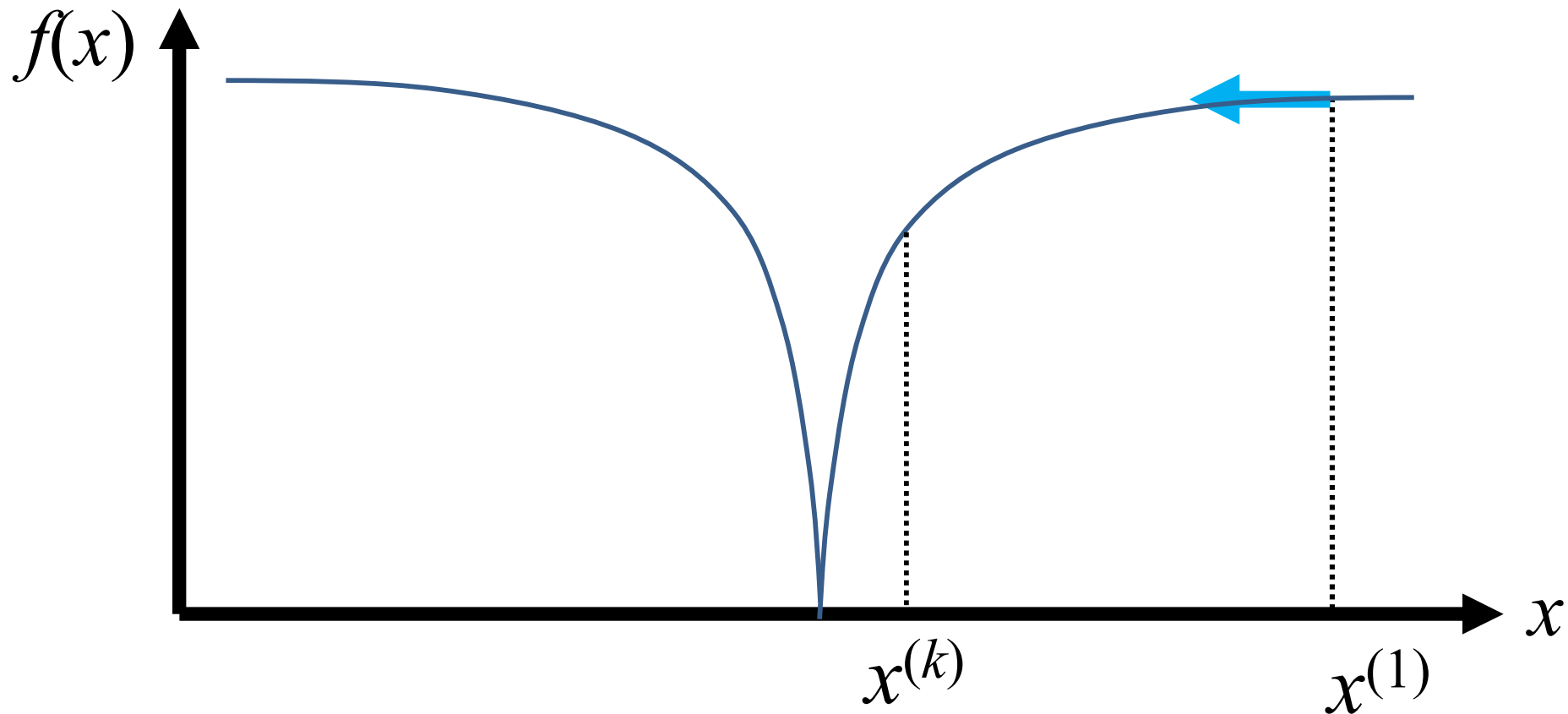


Gradient Descent (Algorithm: Constant Step Size)

$$x^{(k+1)} = x^{(k)} - \alpha \frac{df}{dx} \qquad x^{(k+1)} = x^{(k)} - \alpha^{(k)} \frac{df}{dx}$$

Discussion 1 (your idea): How to specify the constant α ?

Discussion 2 (your idea): How to specify $\alpha^{(k)}$ for each move ?

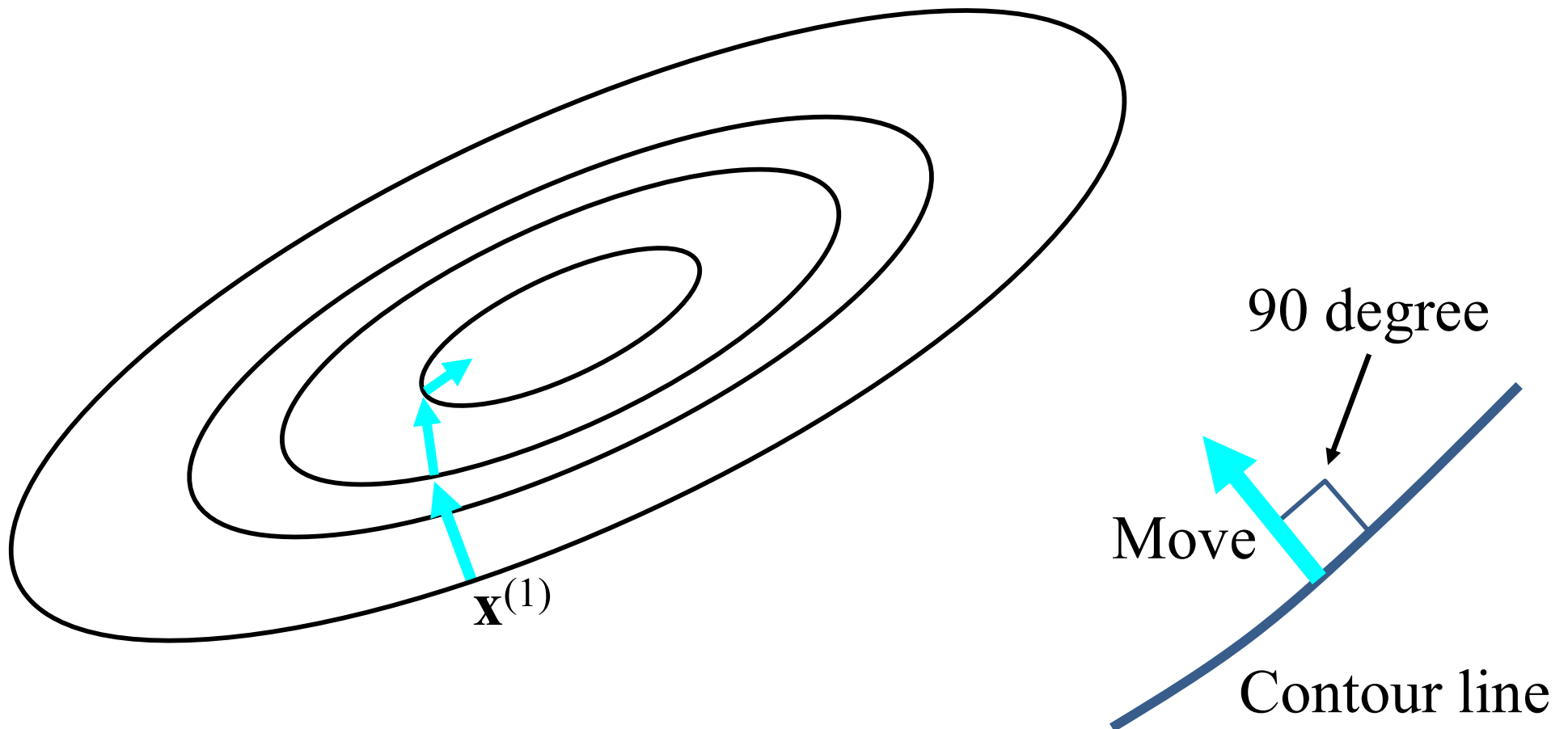


Gradient Descent (Algorithm) (2D)

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha^{(k)} \nabla f(\mathbf{x}^{(k)})$$

$\alpha^{(k)}$: Step size at the k th iteration (positive value).

$$\nabla f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}) \\ \frac{\partial f}{\partial x_2}(\mathbf{x}) \end{pmatrix}$$



Example

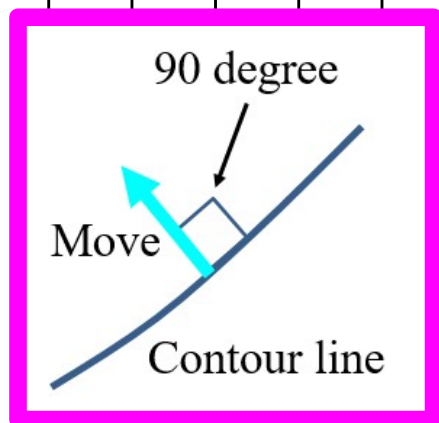
$$f(\mathbf{x}) = x_1^2 + x_2^2$$

$$\nabla f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}) \\ \frac{\partial f}{\partial x_2}(\mathbf{x}) \end{pmatrix} = \begin{pmatrix} 2x_1 \\ 2x_2 \end{pmatrix}$$

$$\mathbf{x} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \Rightarrow \nabla f(\mathbf{x}) = \begin{pmatrix} 2 \\ 0 \end{pmatrix}$$

$$\mathbf{x} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \Rightarrow \nabla f(\mathbf{x}) = \begin{pmatrix} 0 \\ 2 \end{pmatrix}$$

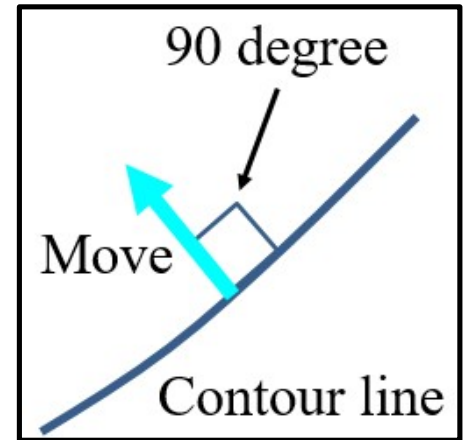
$$\mathbf{x} = \begin{pmatrix} -1/\sqrt{2} \\ -1/\sqrt{2} \end{pmatrix} \Rightarrow \nabla f(\mathbf{x}) = \begin{pmatrix} -2/\sqrt{2} \\ -2/\sqrt{2} \end{pmatrix}$$



Gradient Descent (Algorithm: Constant Step Size)

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha \nabla f(\mathbf{x}^{(k)})$$

α : Step size (positive constant).



Lab Session Task 1

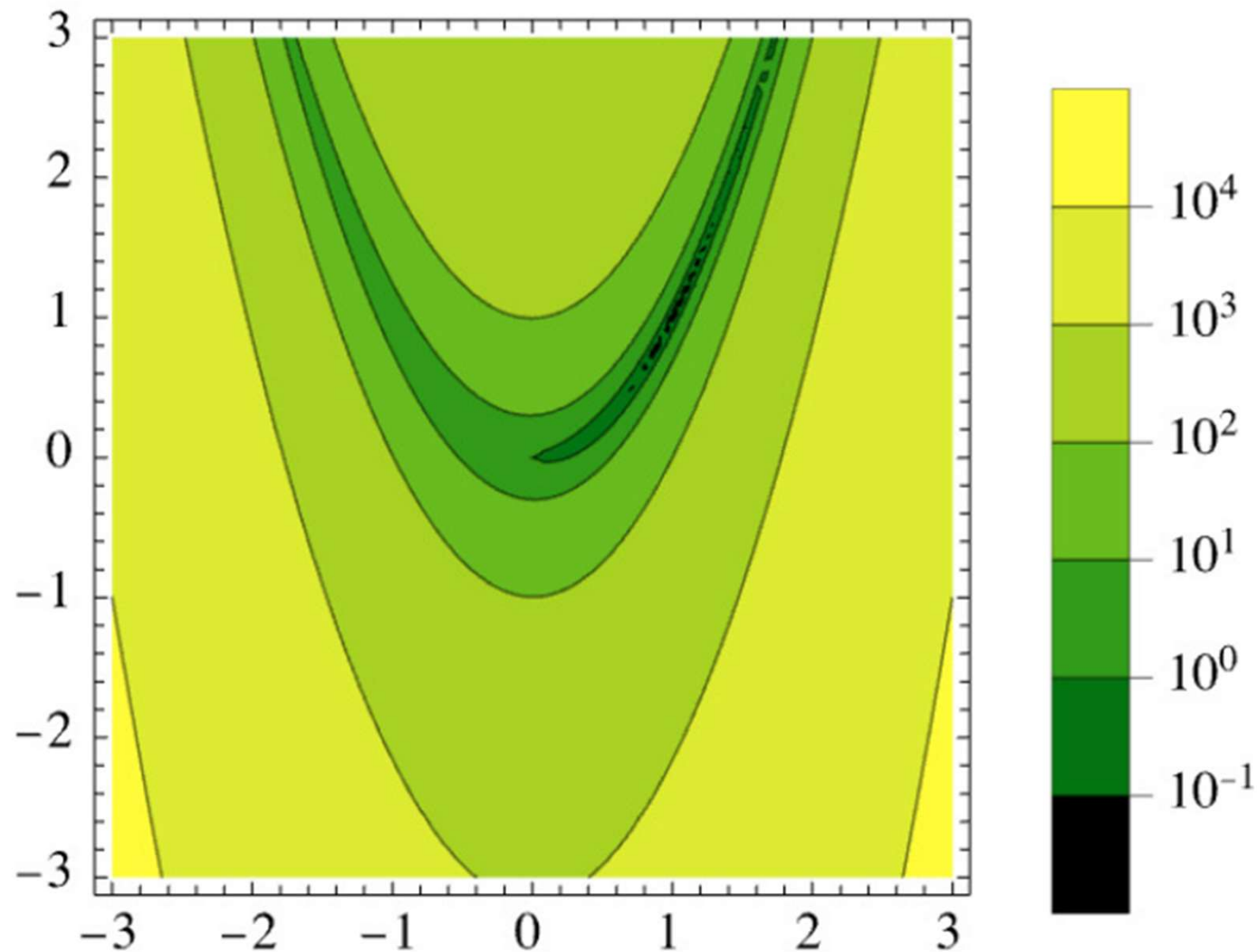
Choose an initial solution in $[-5, 5] \times [-5, 5]$, specify the step size, and show a sequence of moves from the initial solution using the gradient decent algorithm with the constant step size for the following problem (see below). By iterating these steps using a different initial solution and a different step size, show several sequences of moves. Then, choose the most appropriate step size for this problem. That is, your task is to clearly show the search behavior and to find the best specification of the constant step size.

Minimize $f(\mathbf{x}) = (x_1 + 1)^2 / 9 + (x_2 + 1)^2$

Lab Session Task 2

Do the same as Task 1 for the following problem:

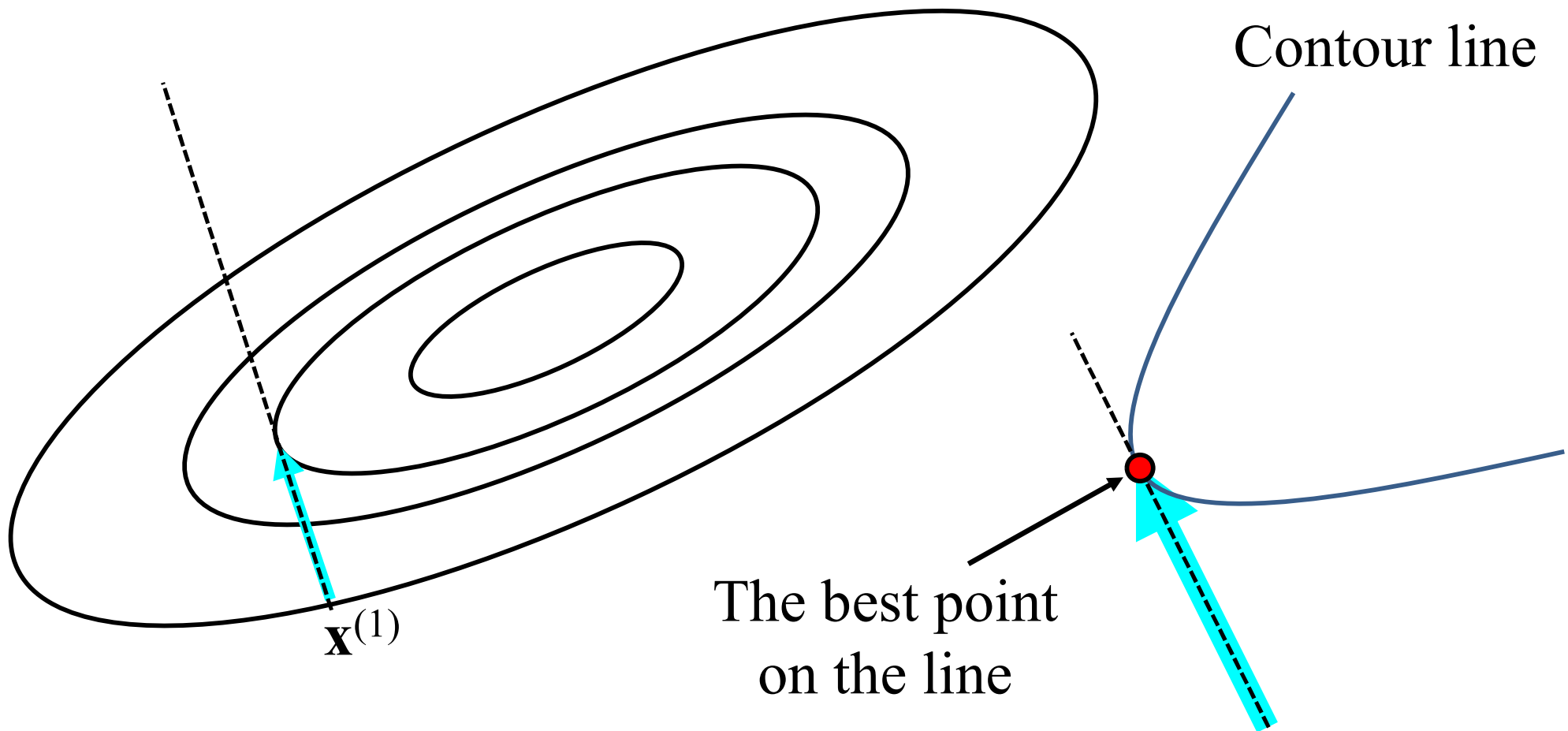
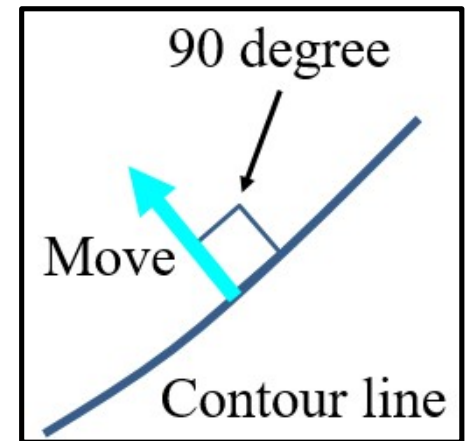
$$\text{Minimize } f(\mathbf{x}) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2$$



Steepest Descent

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha^{(k)} \nabla f(\mathbf{x}^{(k)})$$

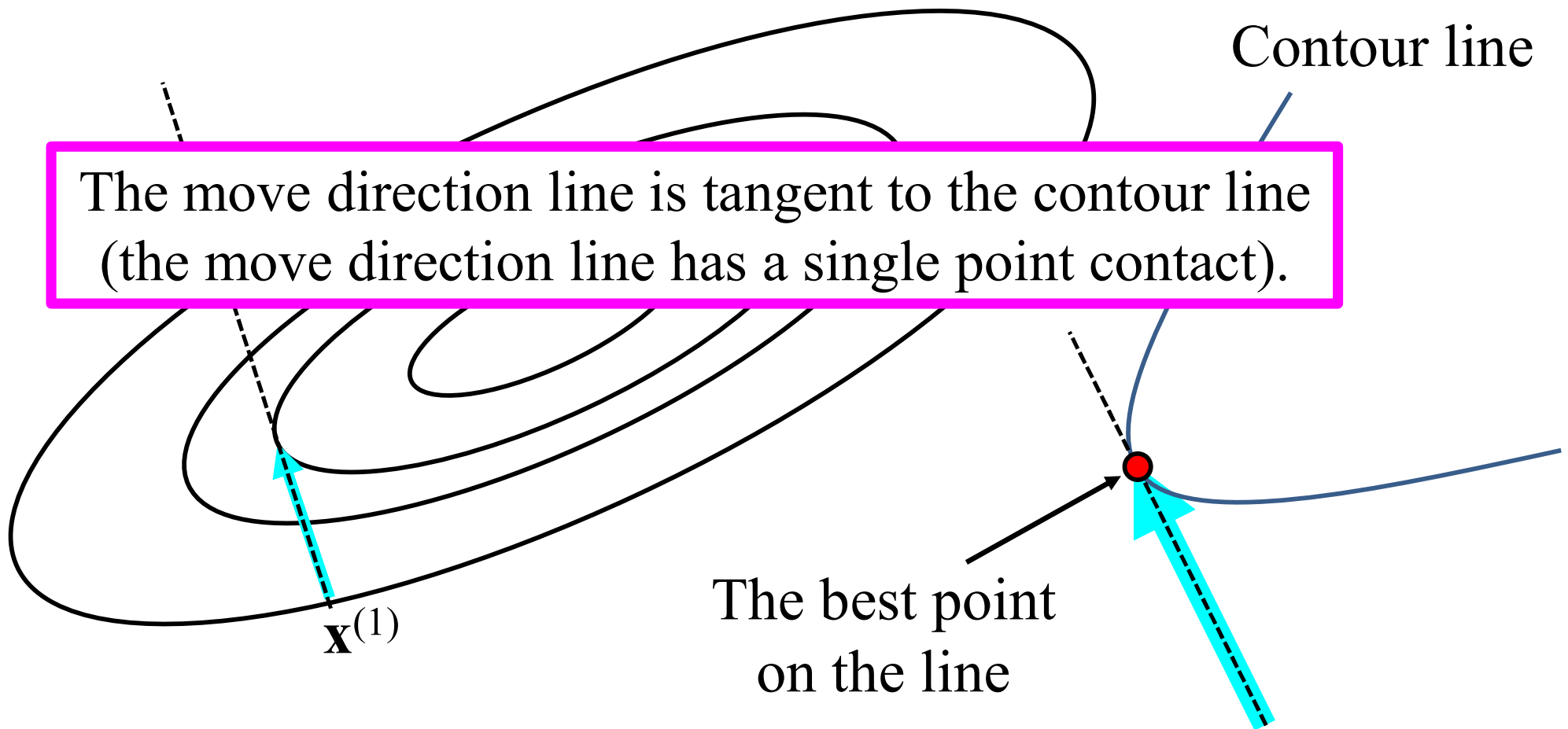
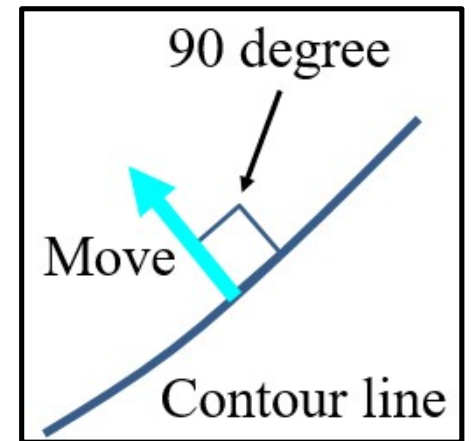
In each iteration, the size of the move is specified so that the objective function is minimized on the move direction line.



Steepest Descent

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha^{(k)} \nabla f(\mathbf{x}^{(k)})$$

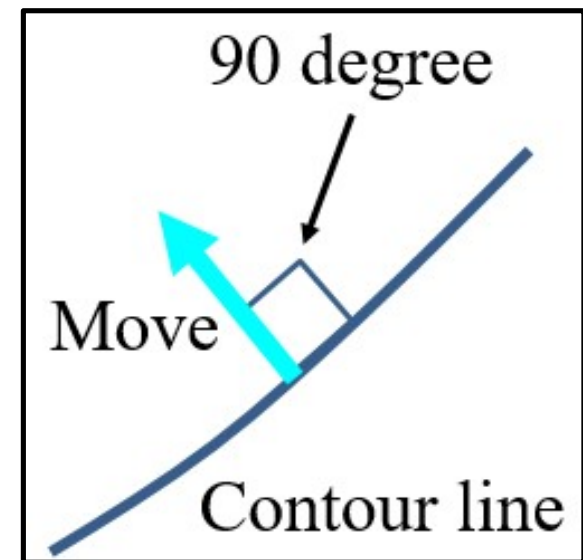
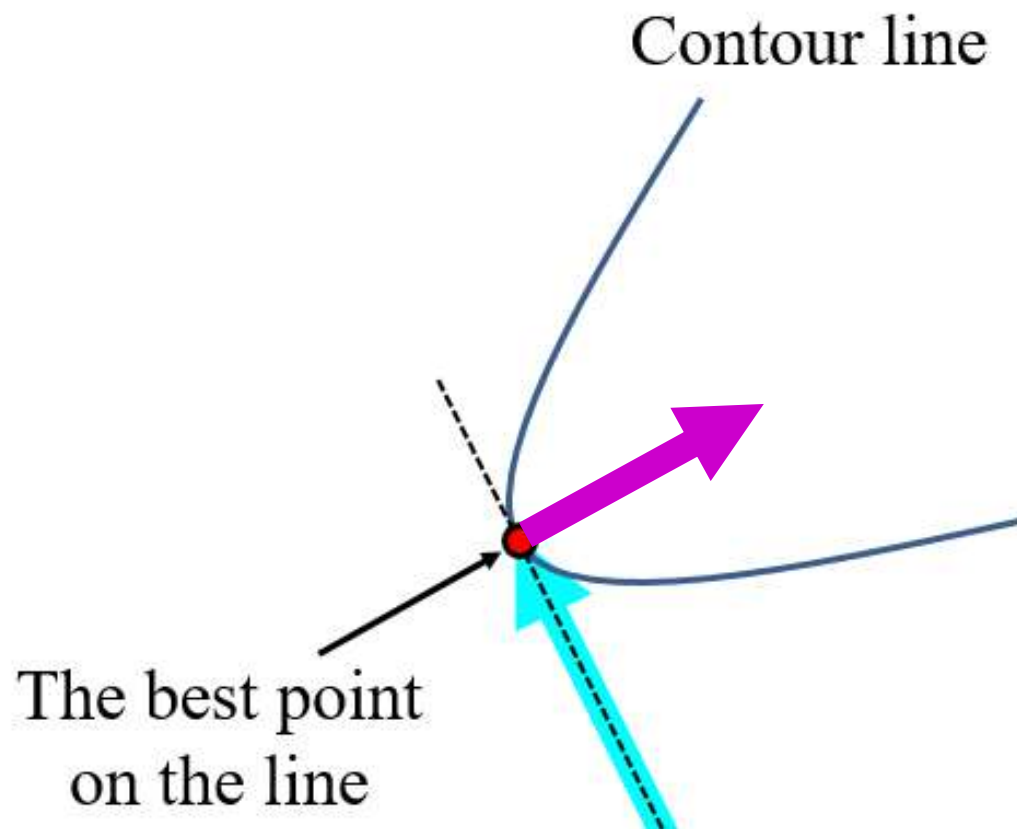
In each iteration, the size of the move is specified so that the objective function is minimized on the move direction line.

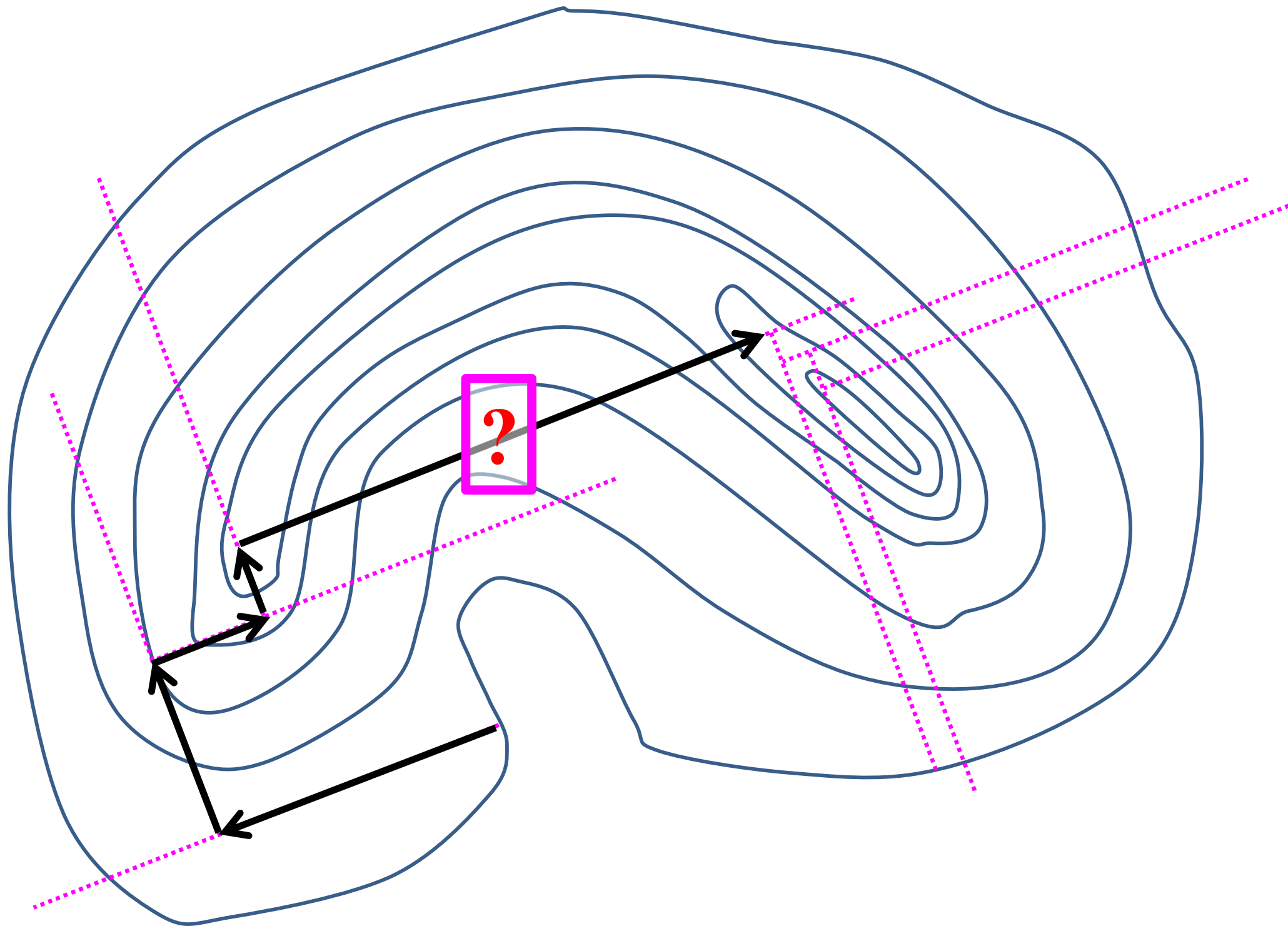


Steepest Descent

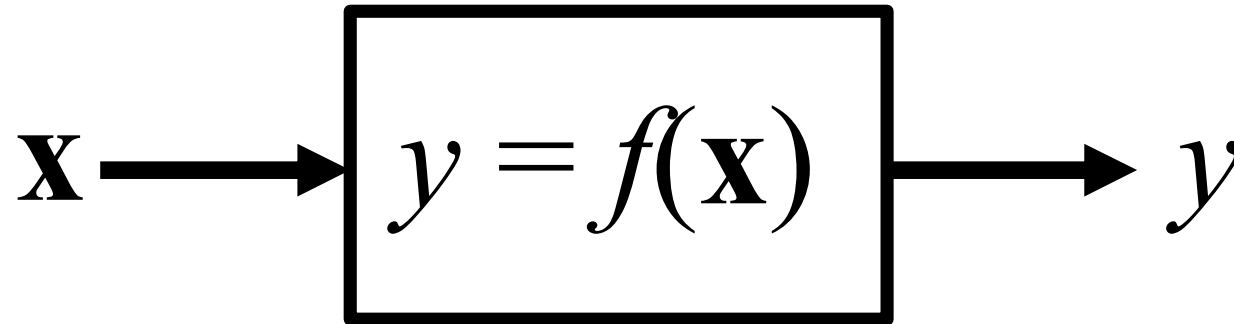
$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha^{(k)} \nabla f(\mathbf{x}^{(k)})$$

Feature: the $(k+1)$ -th move is orthogonal to the k -th move





Black Box Optimization



$\mathbf{x}$: an n -dimensional input vector (real number vector)

y : a real number

$f(\mathbf{x})$: Unknown function

Black Box Optimization Problem:

To search for the optimal solution $\mathbf{x}^*$ which minimizes y .

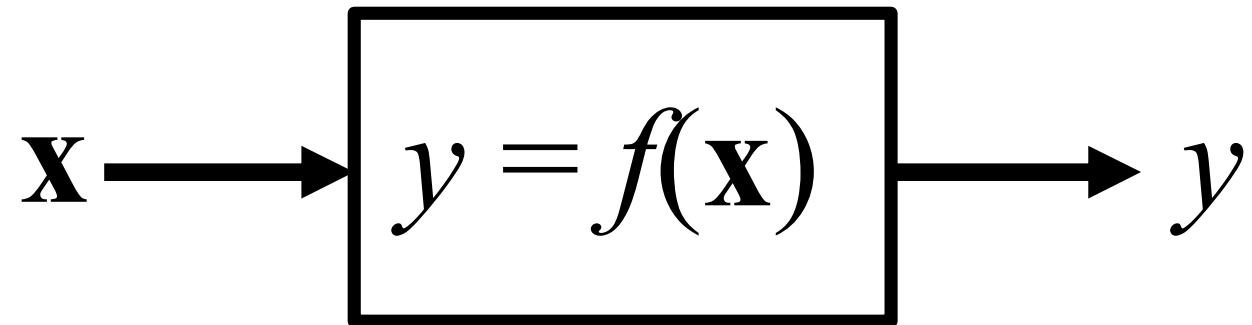
Given computational budget:

You can examine a pre-specified number of solutions.

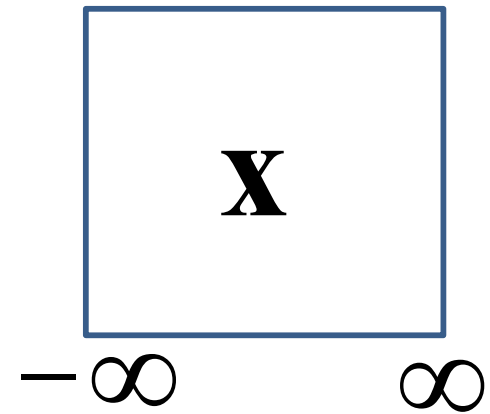
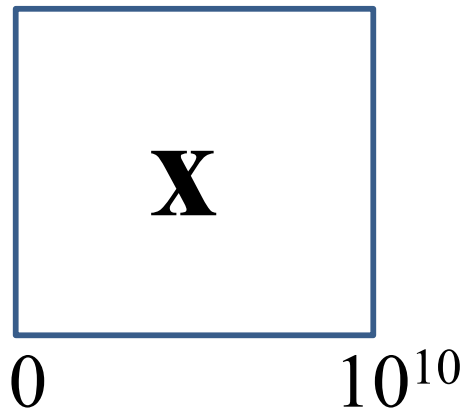
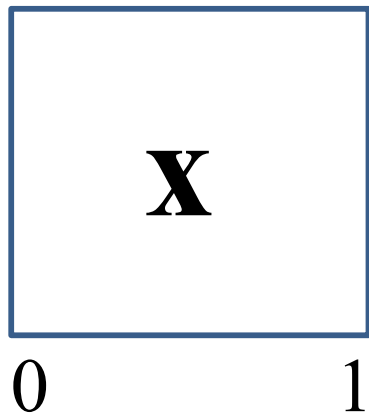
(You can obtain the value of y for those solutions).

Example: You can examine 500 solutions.

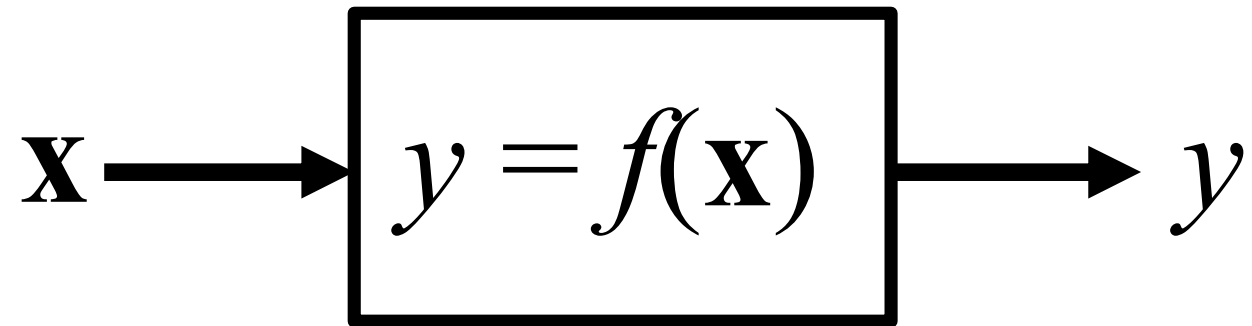
Black Box Optimization



When we have no constraint condition about $\mathbf{x}$, this problem is extremely difficult and unrealistic whereas it is very interesting.

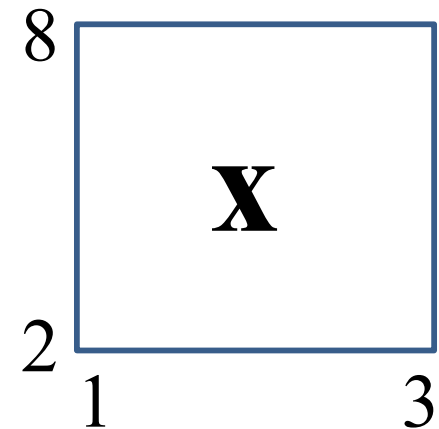


Black Box Optimization

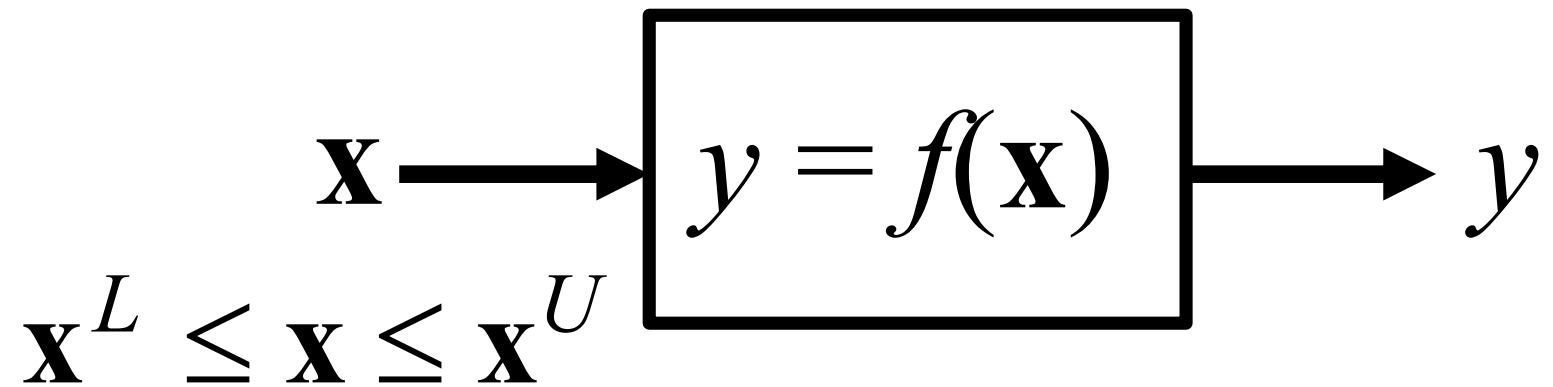


When we have no constraint condition about $\mathbf{x}$, this problem is extremely difficult and unrealistic whereas it is very interesting. Usually, we have the given lower and upper bounds of $\mathbf{x}$.

$$\mathbf{x}^L \leq \mathbf{x} \leq \mathbf{x}^U$$



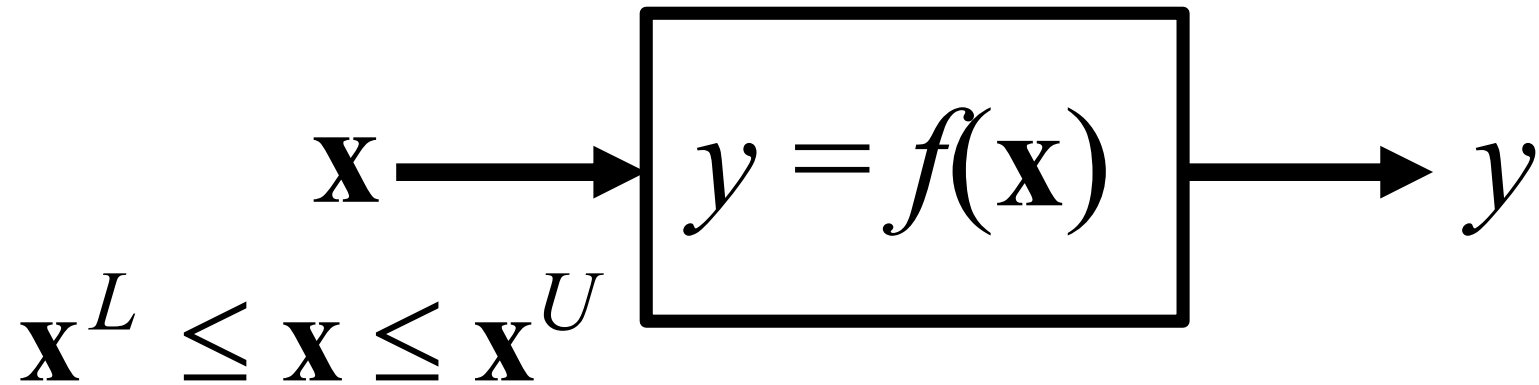
Black Box Optimization



How to solve this problem?

Your idea: _____.

Black Box Optimization



How to solve this problem?

1. Meta-heuristic algorithms such as evolutionary computation.
2. Gradient-based algorithms with gradient estimation by solution sampling in the neighborhood of the current solution.
3. Surrogate model-based algorithms.

Experiment-based Optimization

Minimize $f(\mathbf{x})$ subject to $\mathbf{x} \in X$

where $\mathbf{x}$: Decision vector

X : Feasible region

$f(\mathbf{x})$: Objective function

Main Difficulty:

The calculation (evaluation) of $f(\mathbf{x})$ needs an experiment (simulation), which is time-consuming, expensive, and noisy.

Experiment-based Optimization

Minimize $f(\mathbf{x})$ subject to $\mathbf{x} \in X$

where $\mathbf{x}$: Decision vector

X : Feasible region

$f(\mathbf{x})$: Objective function

Main Difficulty:

The calculation (evaluation) of $f(\mathbf{x})$ needs an experiment (simulation), which is time-consuming, expensive, and noisy.



A noisy optimization problem with a very limited number of solution evaluations (e.g., 100 solution evaluations).

Experiment-based Optimization

Minimize $f(\mathbf{x})$ subject to $\mathbf{x} \in X$

where $\mathbf{x}$: Decision vector

X : Feasible region

$f(\mathbf{x})$: Objective function

Main Difficulty:

The calculation (evaluation) of $f(\mathbf{x})$ needs an experiment (simulation), which is time-consuming, expensive, and noisy.



A noisy optimization problem with a very limited number of solution evaluations (e.g., 100 solution evaluations).

A totally different approach may be needed from standard optimization algorithms (no noise, 1,000,000 evaluations).

Experiment-based Optimization

Minimize $f(\mathbf{x})$ subject to $\mathbf{x} \in X$

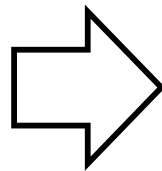
where $\mathbf{x}$: Decision vector

X : Feasible region

$f(\mathbf{x})$: Objective function

The level of an experiment (simulation) is another problem.

Real-World Problem



Optimization Problem

Minimize $f(x)$

Experiment-based Optimization

Minimize $f(\mathbf{x})$ subject to $\mathbf{x} \in X$

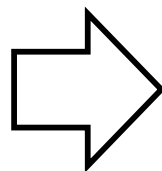
where $\mathbf{x}$: Decision vector

X : Feasible region

$f(\mathbf{x})$: Objective function

The level of an experiment (simulation) is another problem. **If we use a complicated experiment, $f(\mathbf{x})$ may be very accurate but its evaluation needs a long computation time and a large cost. If we use a simple experiment, its result is not accurate.**

Real-World Problem



Optimization Problem

Minimize $f(x)$

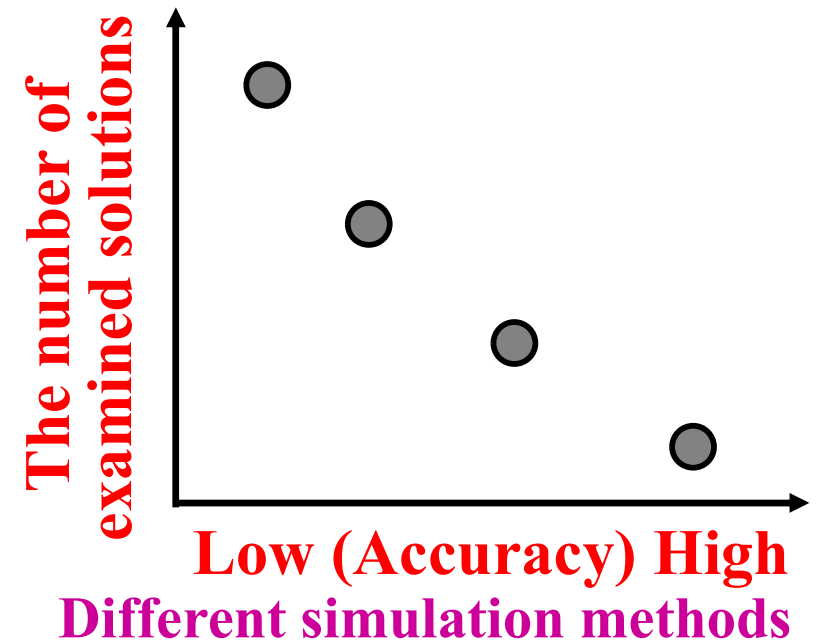
Experiment-based Optimization

Minimize $f(\mathbf{x})$ subject to $\mathbf{x} \in X$

where $\mathbf{x}$: Decision vector

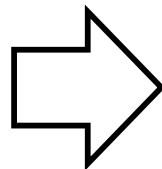
X : Feasible region

$f(\mathbf{x})$: Objective function



The level of an experiment (simulation) is another problem. If we use a complicated experiment, $f(\mathbf{x})$ may be very accurate but its evaluation needs a long computation time and a large cost. If we use a simple experiment, its result is not accurate.

Real-World Problem



Optimization Problem

Minimize $f(\mathbf{x})$

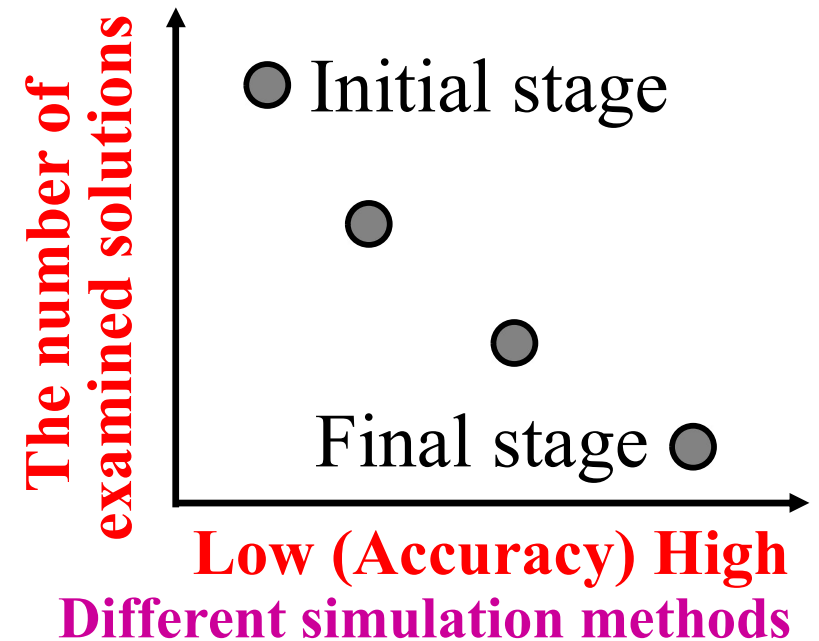
Experiment-based Optimization

Minimize $f(\mathbf{x})$ subject to $\mathbf{x} \in X$

where $\mathbf{x}$: Decision vector

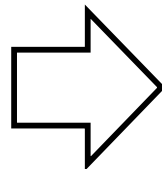
X : Feasible region

$f(\mathbf{x})$: Objective function



The level of an experiment (simulation) is another problem. If we use a complicated experiment, $f(\mathbf{x})$ may be very accurate but its evaluation needs a long computation time and a large cost. If we use a simple experiment, its result is not accurate.

Real-World Problem



Optimization Problem

Minimize $f(\mathbf{x})$

Surrogate-based Optimization

Minimize $f(\mathbf{x})$ subject to $\mathbf{x} \in X$

where $\mathbf{x}$: Decision vector

X : Feasible region

$f(\mathbf{x})$: Objective function

Main Difficulty:

The calculation of $f(\mathbf{x})$ is time-consuming and expensive.

Surrogate-based Optimization

Minimize $f(\mathbf{x})$ subject to $\mathbf{x} \in X$

where $\mathbf{x}$: Decision vector

X : Feasible region

$f(\mathbf{x})$: Objective function

Main Difficulty:

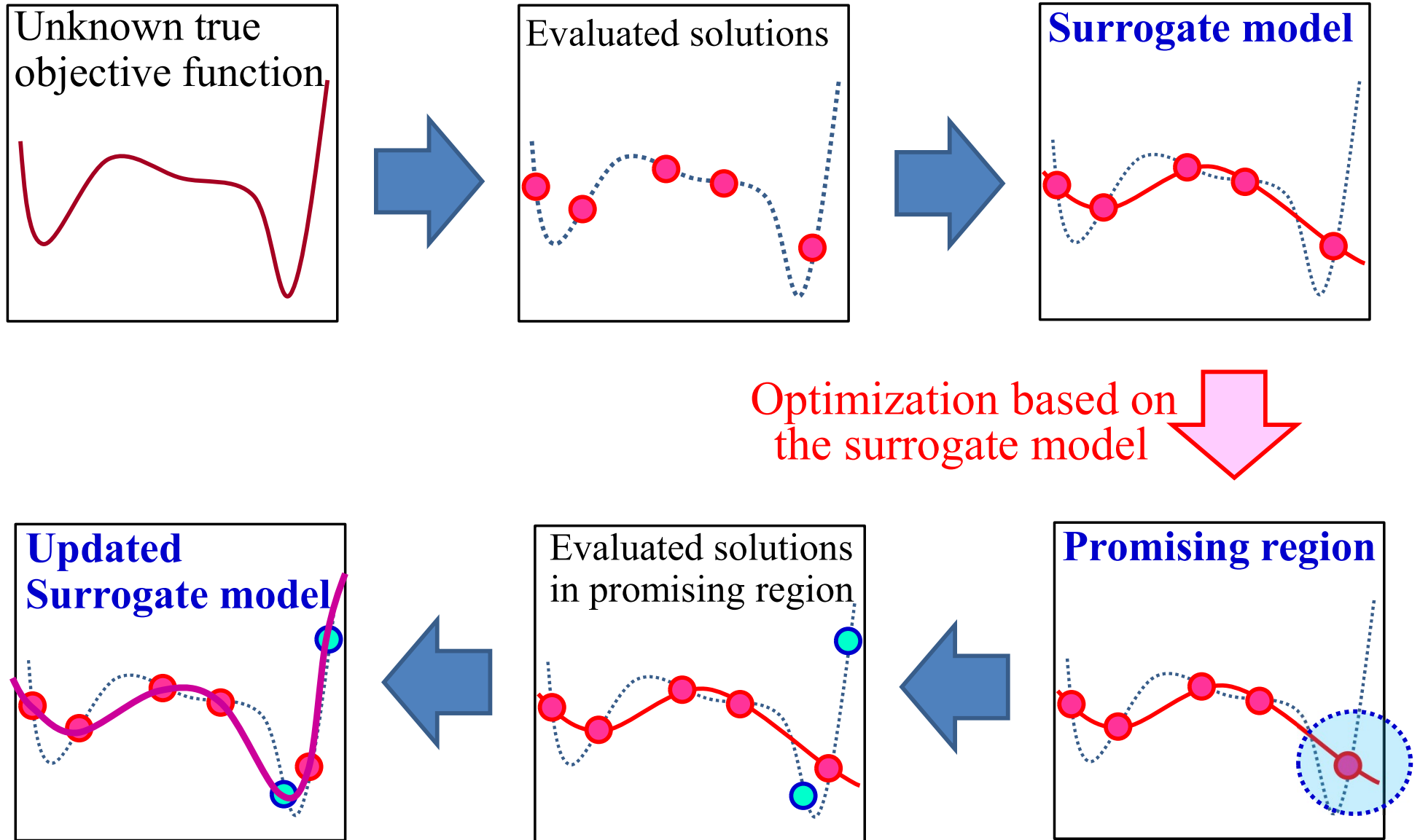
The calculation of $f(\mathbf{x})$ is time-consuming and expensive.



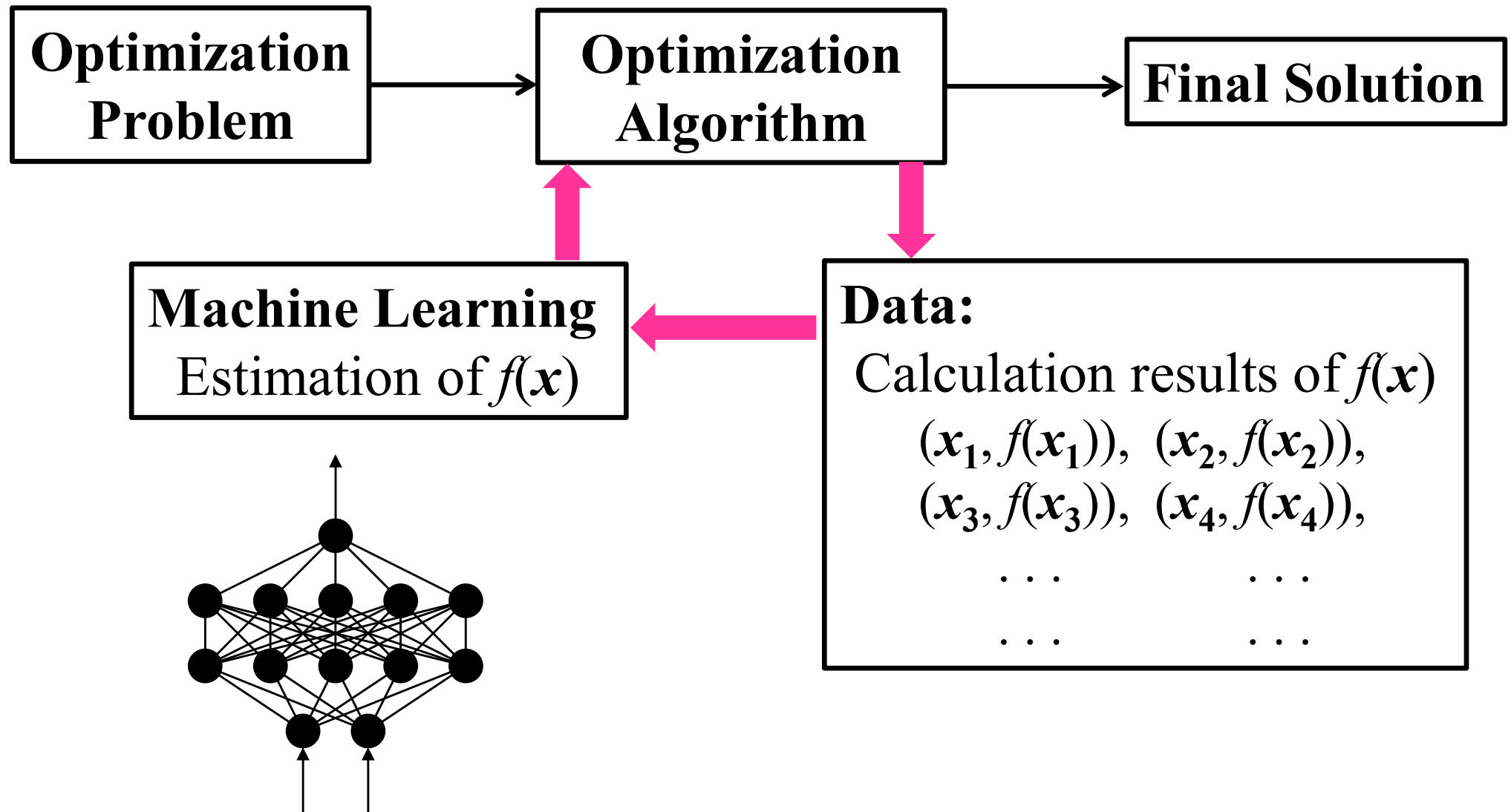
During the search (e.g., by evolutionary computation):

- $f(\mathbf{x})$ is calculated for only a small number of solutions.
- $f(\mathbf{x})$ is estimated for other solutions using the calculated solutions (using a machine learning method such as neural networks).

Surrogate-based Optimization

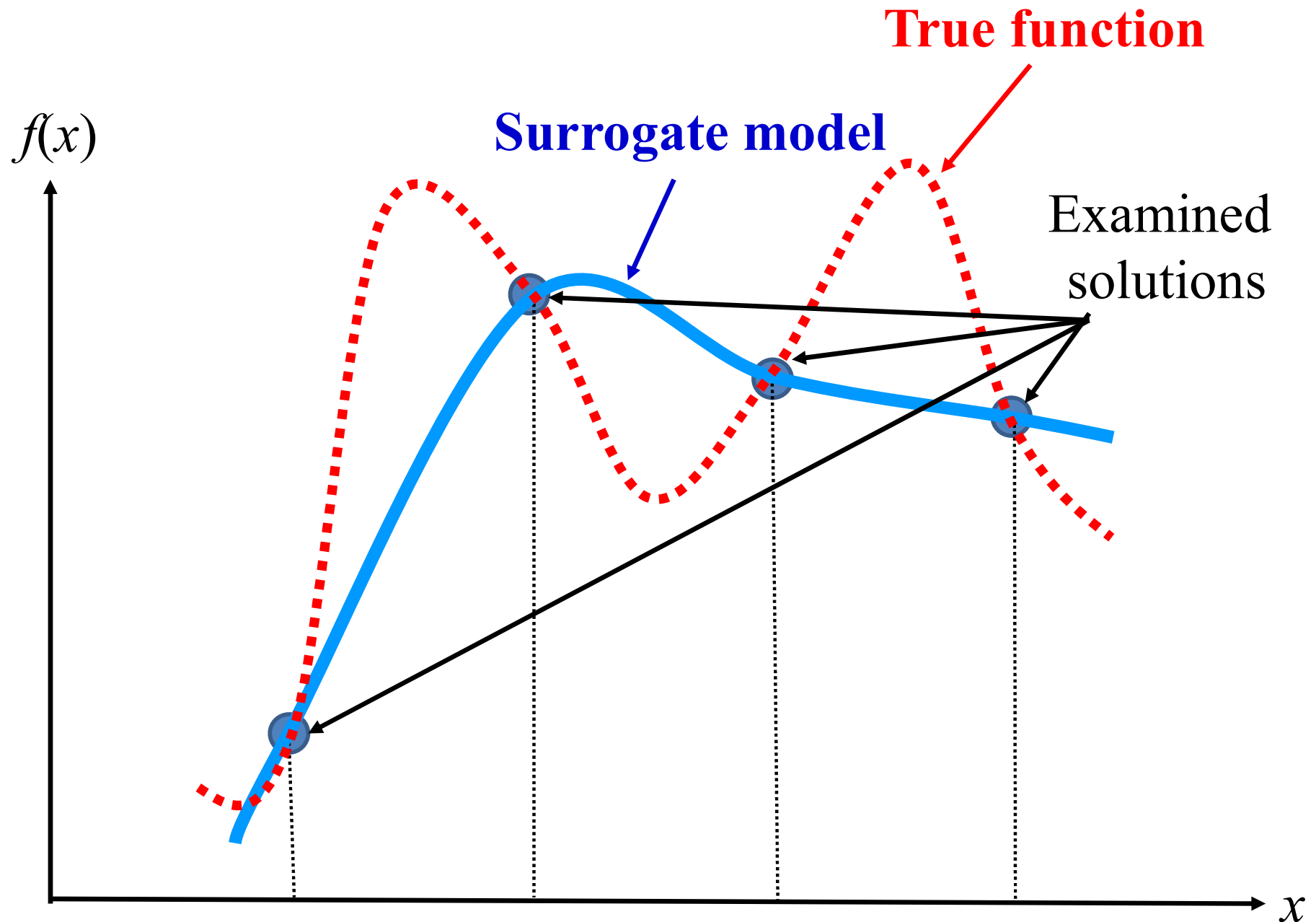


Online Data-Driven Optimization

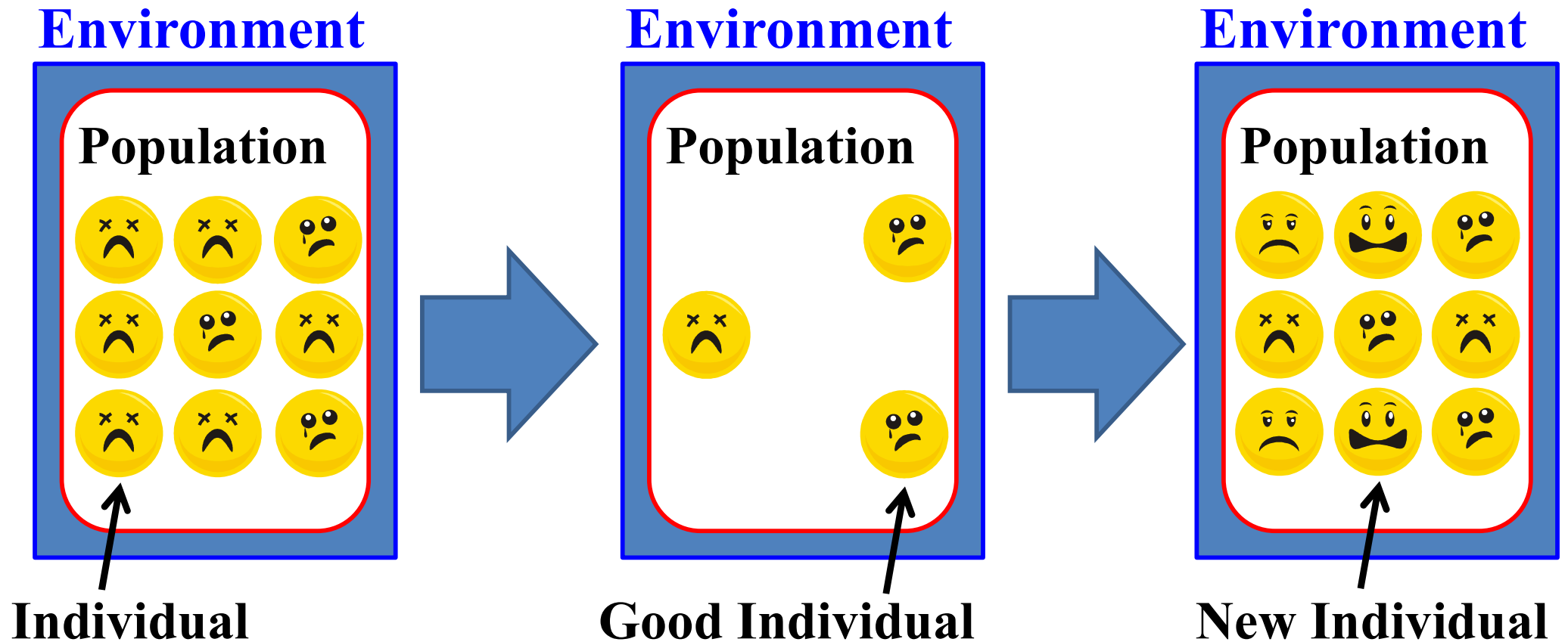


Surrogate Model

Surrogate Model

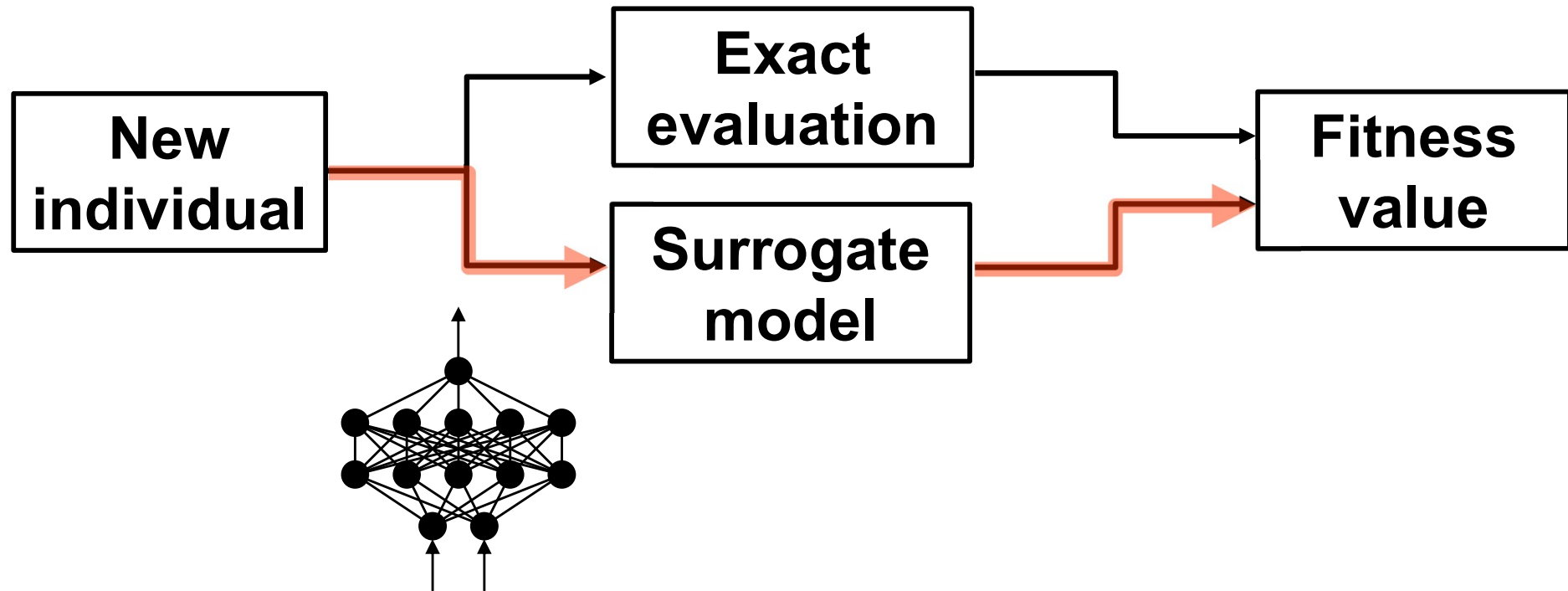


Basic Idea of Evolutionary Computation



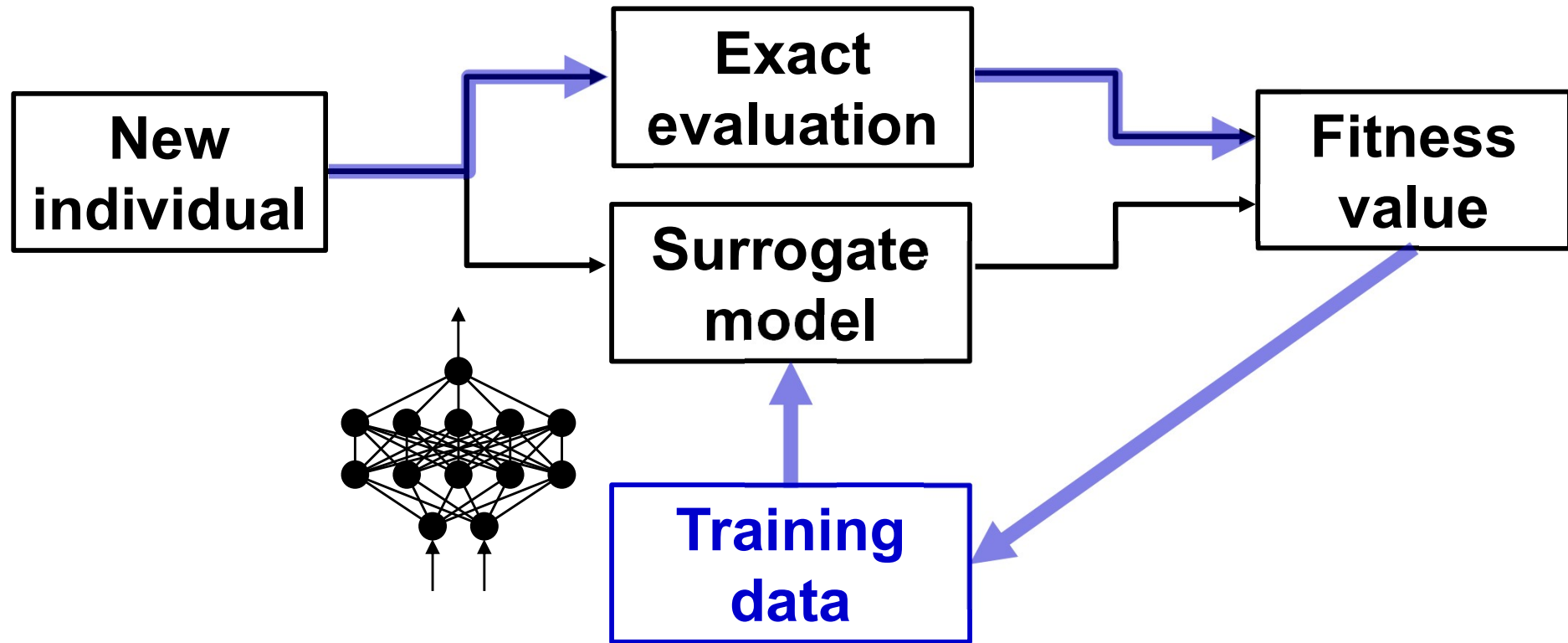
- (1) A population of individuals is randomly generated.
- (2) Each individual is evaluated in the environment. **Surrogate**
- (3) Good individuals survive probabilistically.
- (4) New individuals are generated from the good individuals.

Surrogate Model



Basic Idea: To decrease the computation load for fitness evaluation by using a surrogate model.

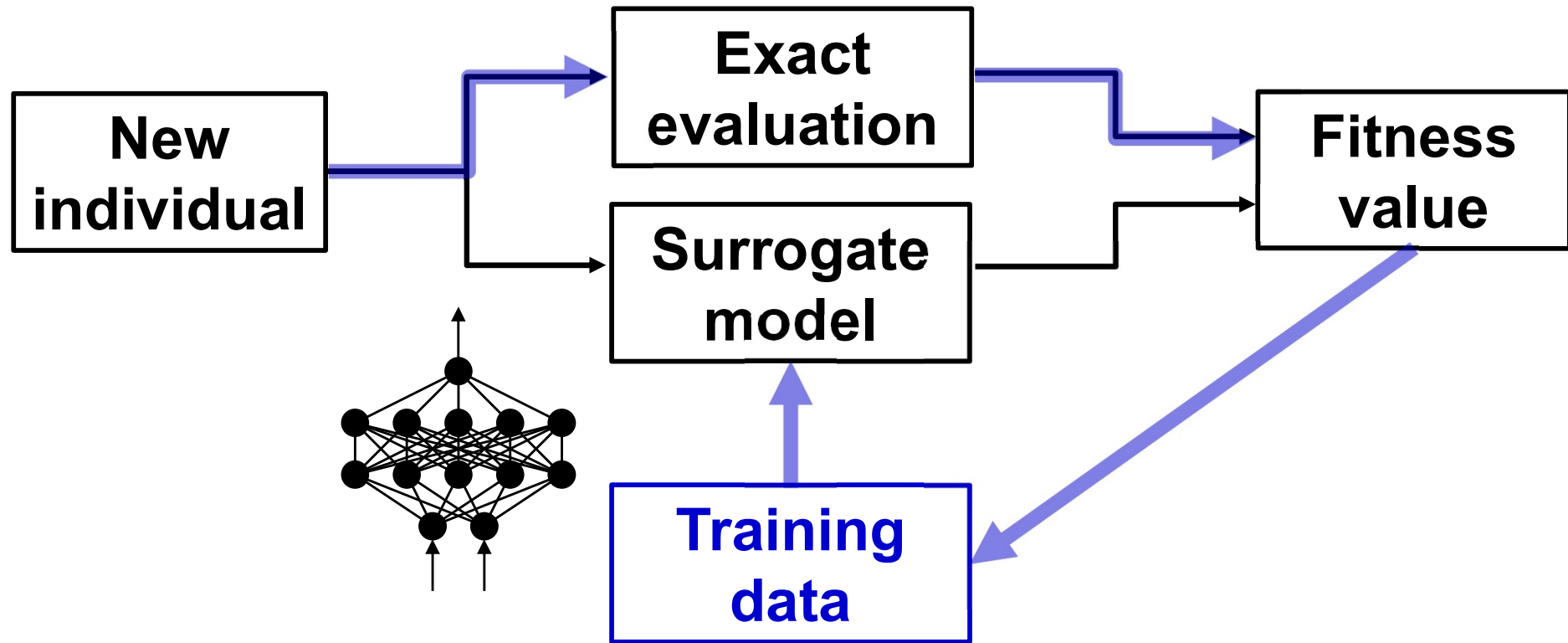
Data Collection and Model Building



1st Step: Collect training data by evaluating some solutions.

2nd Step: Train a surrogate model. This step is a machine learning problem to train a function approximator (i.e., nonlinear modelling).

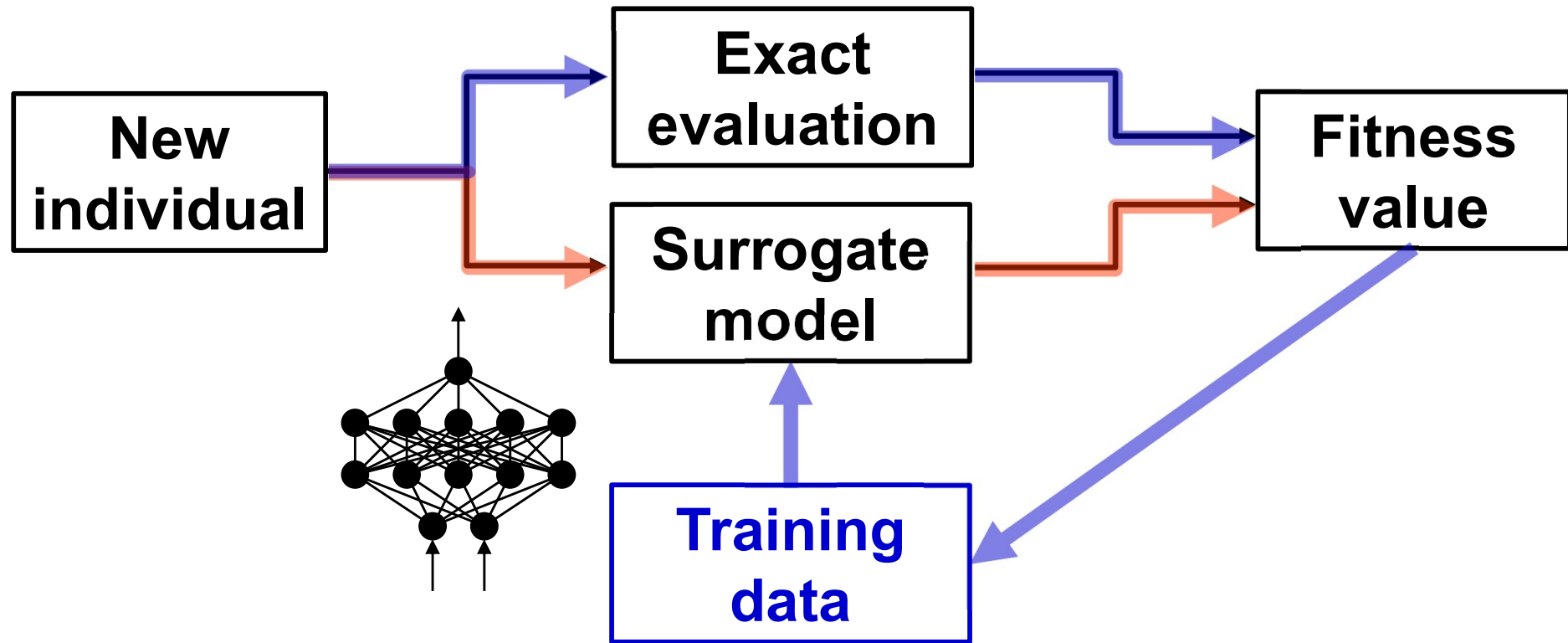
Data Collection and Model Building



Q1: Which method should be used for function approximation (e.g., multi-layer feedforward neural networks, RBF networks, fuzzy systems, polynomial models, ...).

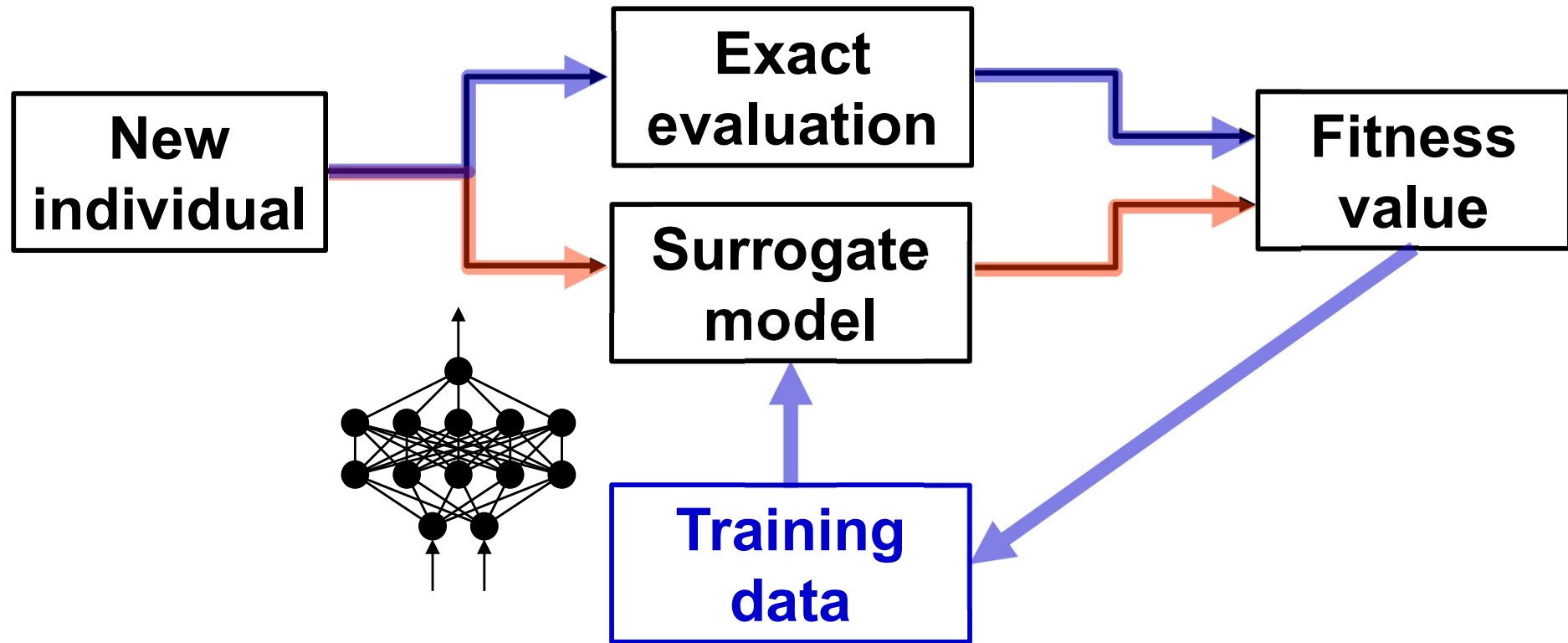
Q2: How many solutions are needed as training data?

Use of Surrogate Model



The trained surrogate model is used for fitness evaluation of each individual. However, the exact evaluation of the fitness value is also periodically performed for updating (re-training) the surrogate model in an online manner. **Q. How often?**

Use of Surrogate Model



The point is how to assign the available computation load for the exact fitness evaluation to different phases of search. One extreme strategy is to assign it only to the initial phase. Another strategy is to evenly assign it to the entire search process.

Examples (when 200 solutions can be exactly evaluated):

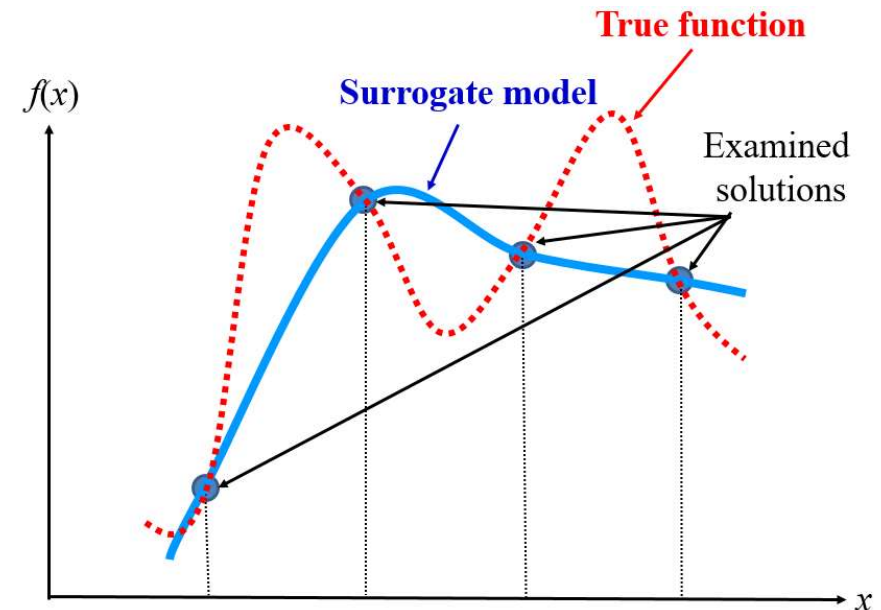
Implementation 1:

Use an optimization algorithm under the termination condition of 200 solution evaluations (e.g., local search with 200 solution evaluations, 10 generations of GA with population size 20, 4 generations of GA with population size 50, random creation of 200 solutions). Then train a surrogate model using the evaluated 200 solutions. After that, an algorithm is applied to the surrogate model. The final solution is the solution with the best objective value based on the surrogate model.

Q. Is this a good implementation? What is the main difficulty in this implementation?



First 200 solutions are evaluated.



Examples (when 200 solutions can be exactly evaluated):

Implementation 2:

Use an optimization algorithm under the termination condition of 100 solution evaluations. Then train a surrogate model using the evaluated 100 solutions. After that, an algorithm is applied to the surrogate model. Every 100 new solutions, one solution is evaluated. When a solution is evaluated, the surrogate model is updated. The objective function values of the other 99 solutions are estimated by the updated surrogate model. In this manner, $100 + 100 \times 100$ solutions are examined (i.e., evaluated or estimated). The final solution is the best solution among the actually evaluated solutions.

Q. Is this a good implementation? What is the main difficulty?



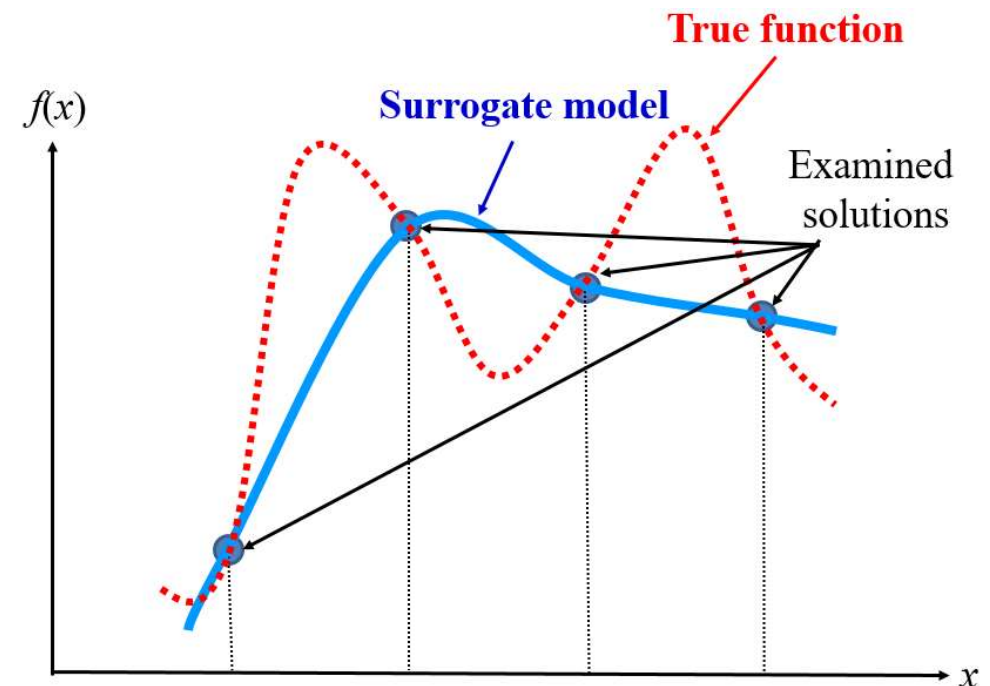
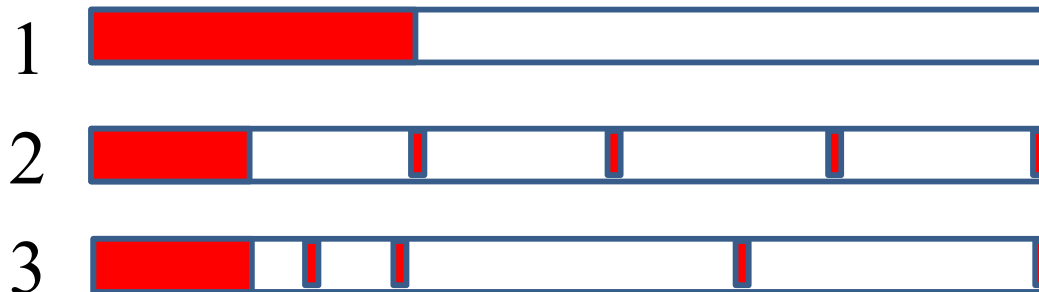
3

Examples (when 200 solutions can be exactly evaluated):

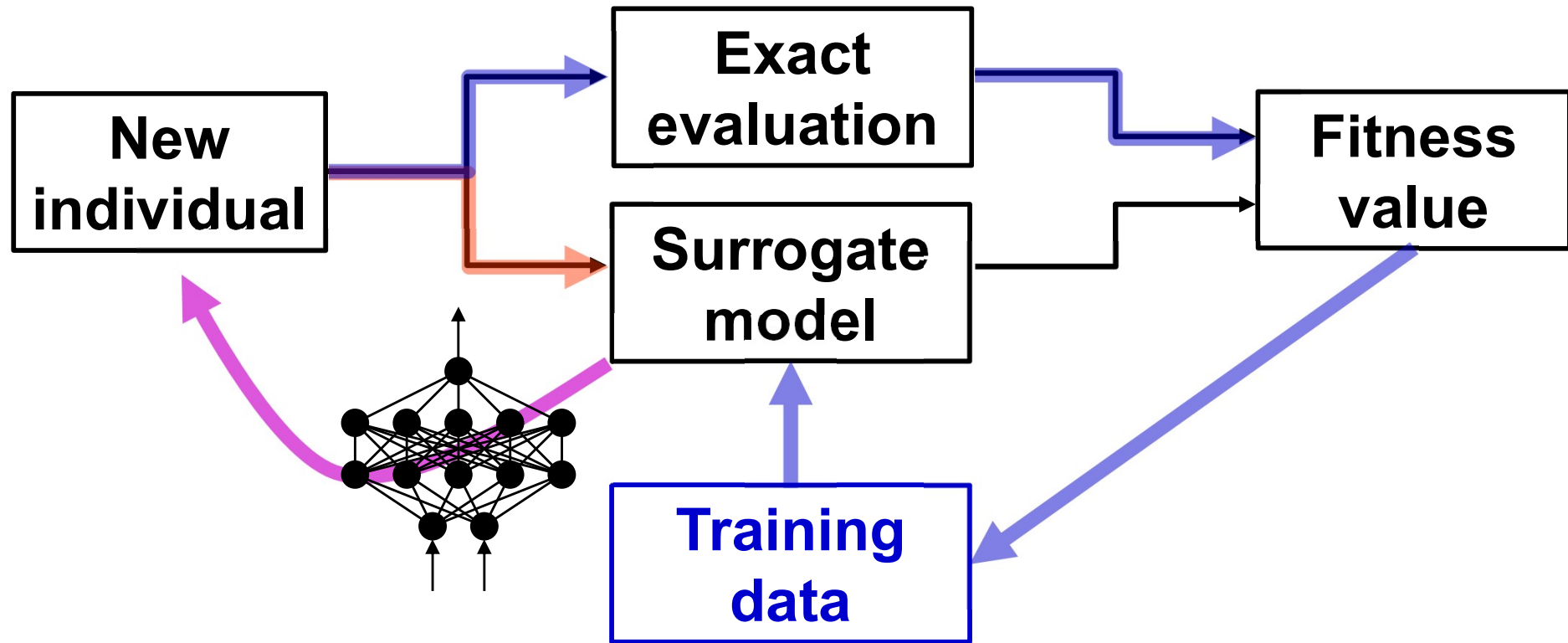
Implementation 3:

This is almost the same as Implementation 2. After the surrogate model is obtained, a solution is actually evaluated only when a better solution (based on the surrogate model) than the current best solution (based on the actual evaluation) is found.

Q. Is this a good implementation? What is the main difficulty?

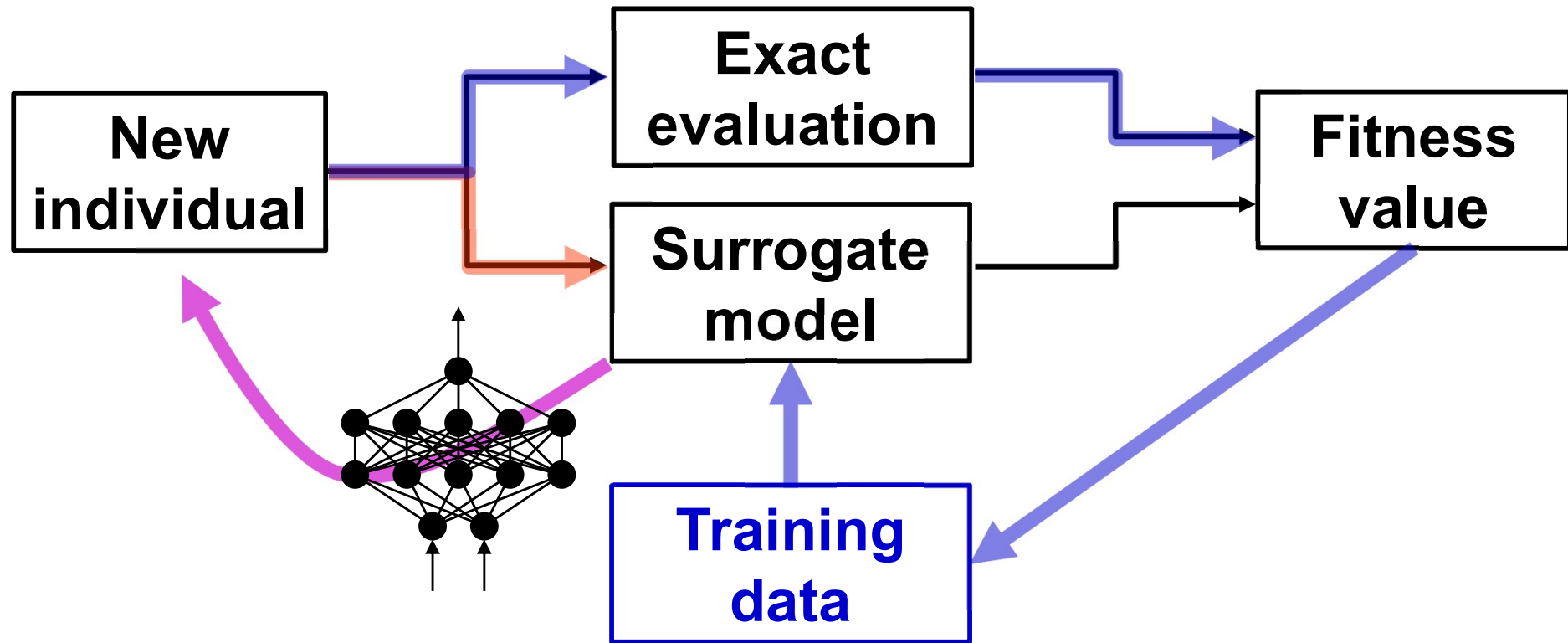


Surrogate Model for Solution Selection



The surrogate model can be used for choosing a solution to be exactly evaluated. For example, 100 solutions are evaluated using the surrogate model. Then one solution is selected for the exact evaluation.

Surrogate Model for Solution Selection



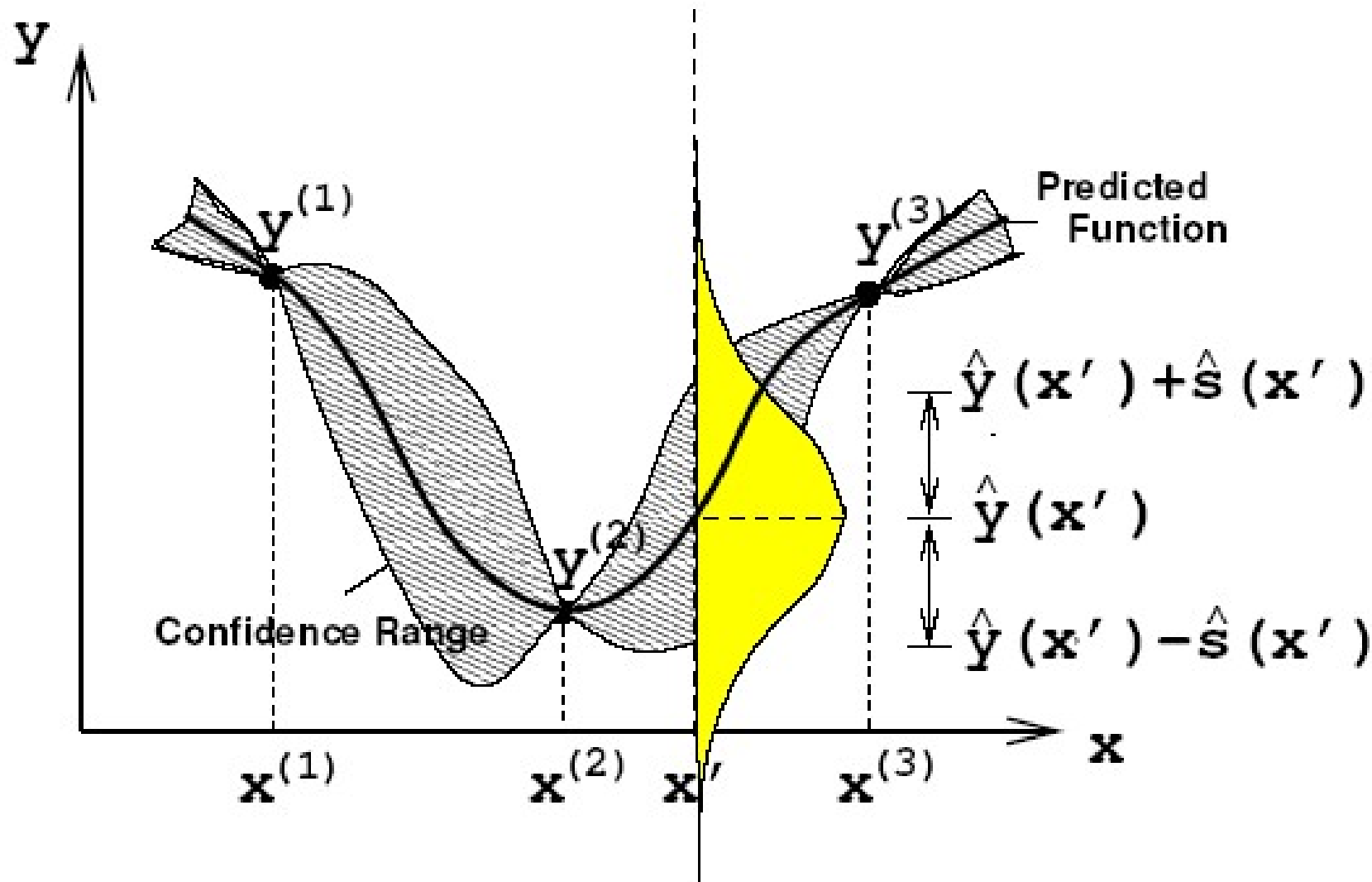
Q. How to choose a small number of solutions to be exactly evaluated from a large number of candidate solutions which have been evaluated by the surrogate model.

Surrogate Model for Solution Selection

Q. How to choose a small number of solutions to be exactly evaluated from a large number of solutions which have been evaluated by the surrogate model.

1. The solution which has the highest estimated objective value by the surrogate model.
2. The solution which is the most isolated from the evaluated solutions (i.e., with no evaluated solutions in the neighborhood).
3. The best solution by a combined criterion of 1 and 2.

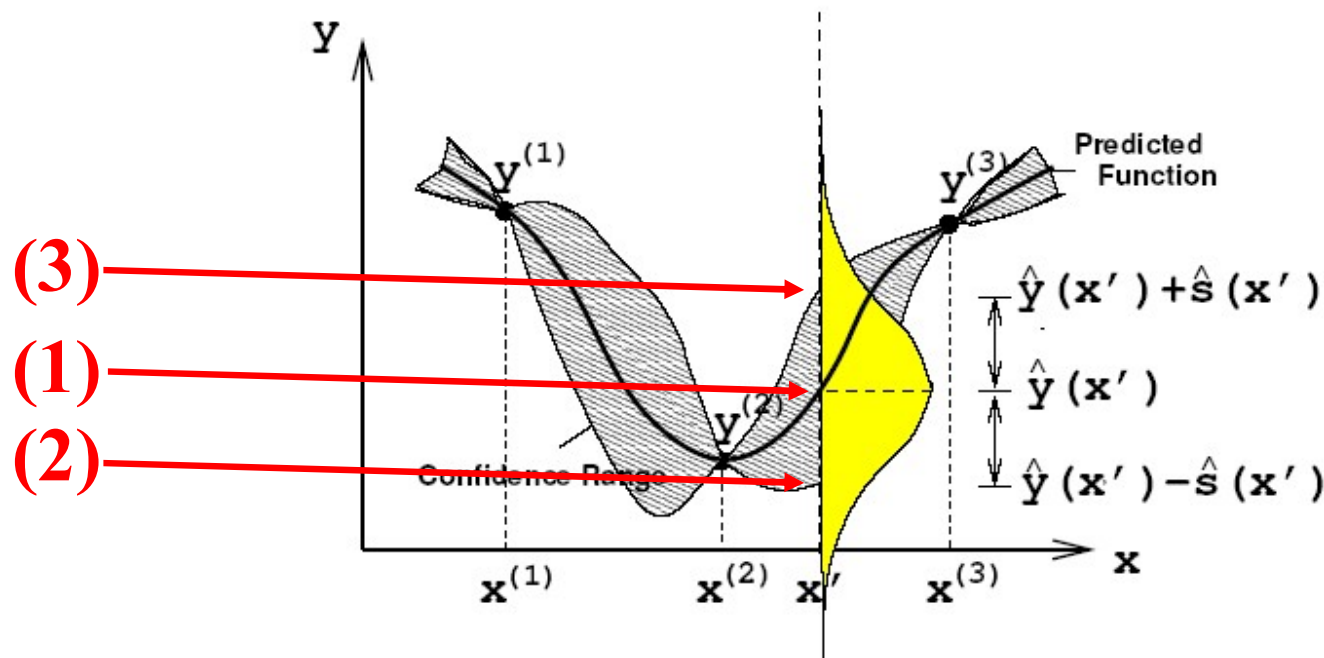
When a stochastic model is used for function approximation (i.e., when an estimated objective function value is a probability distribution or an interval):



Taken from Emmerich et al, 2005

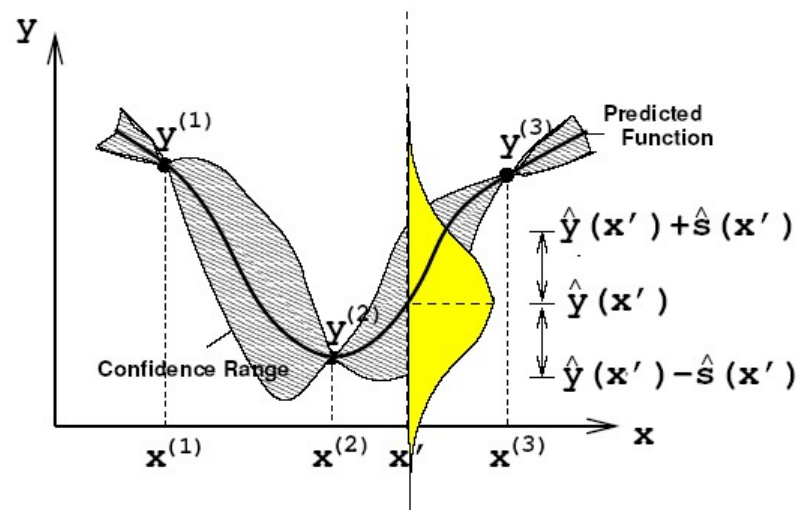
When a stochastic model is used for function approximation (i.e., when an estimated objective function value is a probability distribution or an interval):

1. The solution which has the best expected objective function value by the surrogate model (i.e., smallest center value).
2. The solution which has the best optimistic value by the surrogate model (i.e., smallest best possible value).
3. The solution which has the best pessimistic value by the surrogate model (i.e., smallest worst possible value).



When a stochastic model is used for function approximation (i.e., when an estimated objective function value is a probability distribution or an interval):

1. The solution which has the best expected objective function value by the surrogate model (i.e., smallest center value).
2. The solution which has the best optimistic value by the surrogate model (i.e., smallest best possible value).
3. The solution which has the best pessimistic value by the surrogate model (i.e., smallest worst possible value).
4. The solution which has the largest uncertainty.



Surrogate Model Evaluation

Q. How to evaluate the surrogate model?

Given: A training data set $\{(\mathbf{x}_i, y_i) \mid i = 1, 2, \dots, N\}$

A function approximator (e.g., a neural network)

Surrogate Model Evaluation

Q. How to evaluate the surrogate model?

Given: A training data set $\{(\mathbf{x}_i, y_i) \mid i = 1, 2, \dots, N\}$

A function approximator (e.g., a neural network)

1. Use of the mean square error for testing data in the cross-validation method:

$$\{(y_1 - f(\mathbf{x}_1))^2 + (y_2 - f(\mathbf{x}_2))^2 + \dots\} / |\text{Testing Data}|$$

Example: 10-fold cross-validation (10-CV): The given training data are randomly divided into ten subsets of the same size. One subset is used as testing data and the other are used as training data. A model is trained by the training data and evaluated by the testing data. This is iterated 10 times for the current data set subdivision (and iterated for other subdivisions).

More straight forward method: Calculate the average errors over the actually evaluated solutions in the optimization process.

Surrogate Model Evaluation

Q. How to evaluate the surrogate model?

Given: A training data set $\{(\mathbf{x}_i, y_i) \mid i = 1, 2, \dots, N\}$

A function approximator (e.g., a neural network)

2. Use of the average accuracy with respect to the comparison of two solutions in testing data in the cross-validation method (i.e., average accuracy with respect to the following question: Which solution is better between solutions $\mathbf{x}_i$ and $\mathbf{x}_j$?)

This can be performed using the actually evaluated solutions in the optimization process.

Q. Which do you think a better evaluation method between the approximation accuracy and the comparison accuracy?

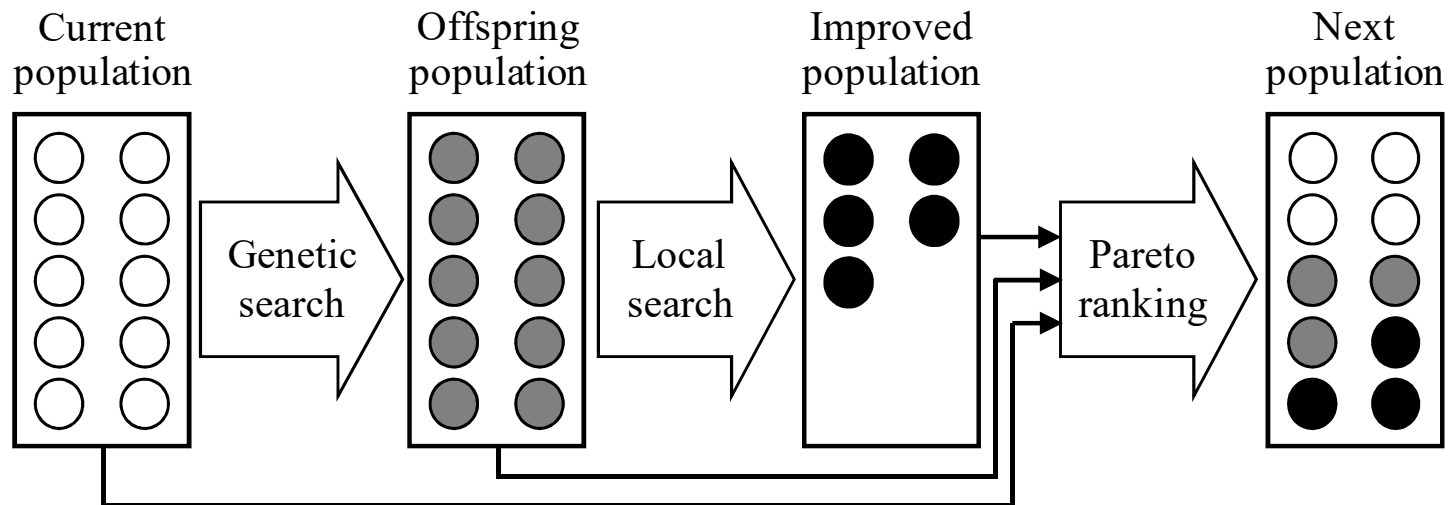
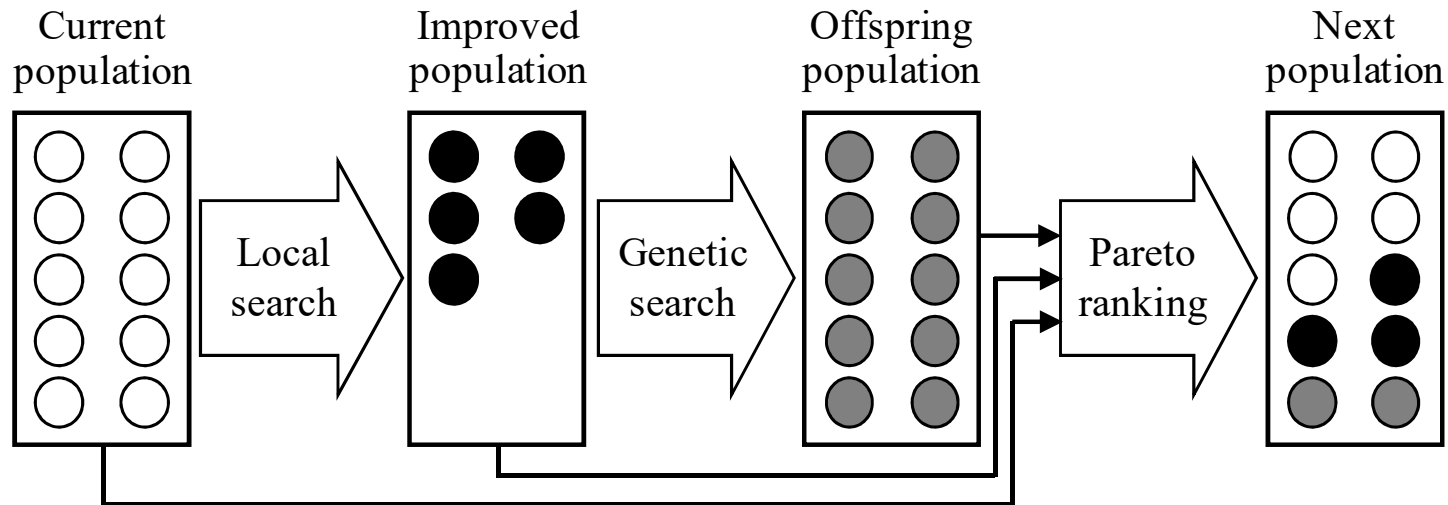
Example of Implementation:

Ensemble-based surrogate model

Multiple surrogate models are used for approximating a single fitness function. The average value or the weighted average value is the final estimated fitness value.

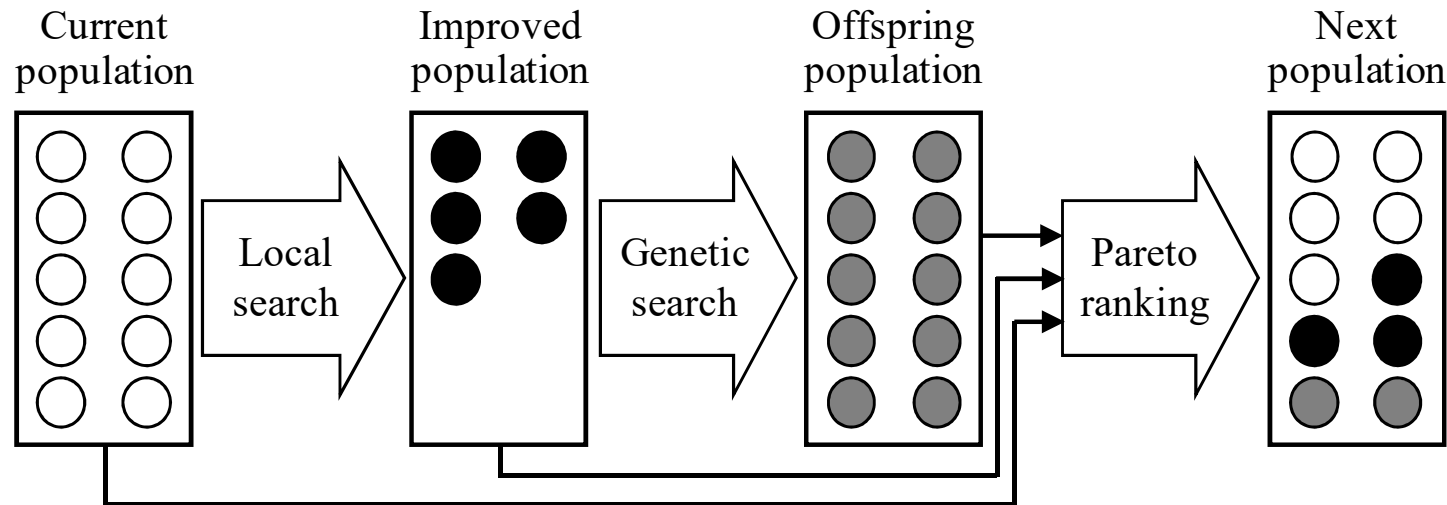
Example of Implementation:

Surrogate-based Memetic Algorithm



Example of Implementation:

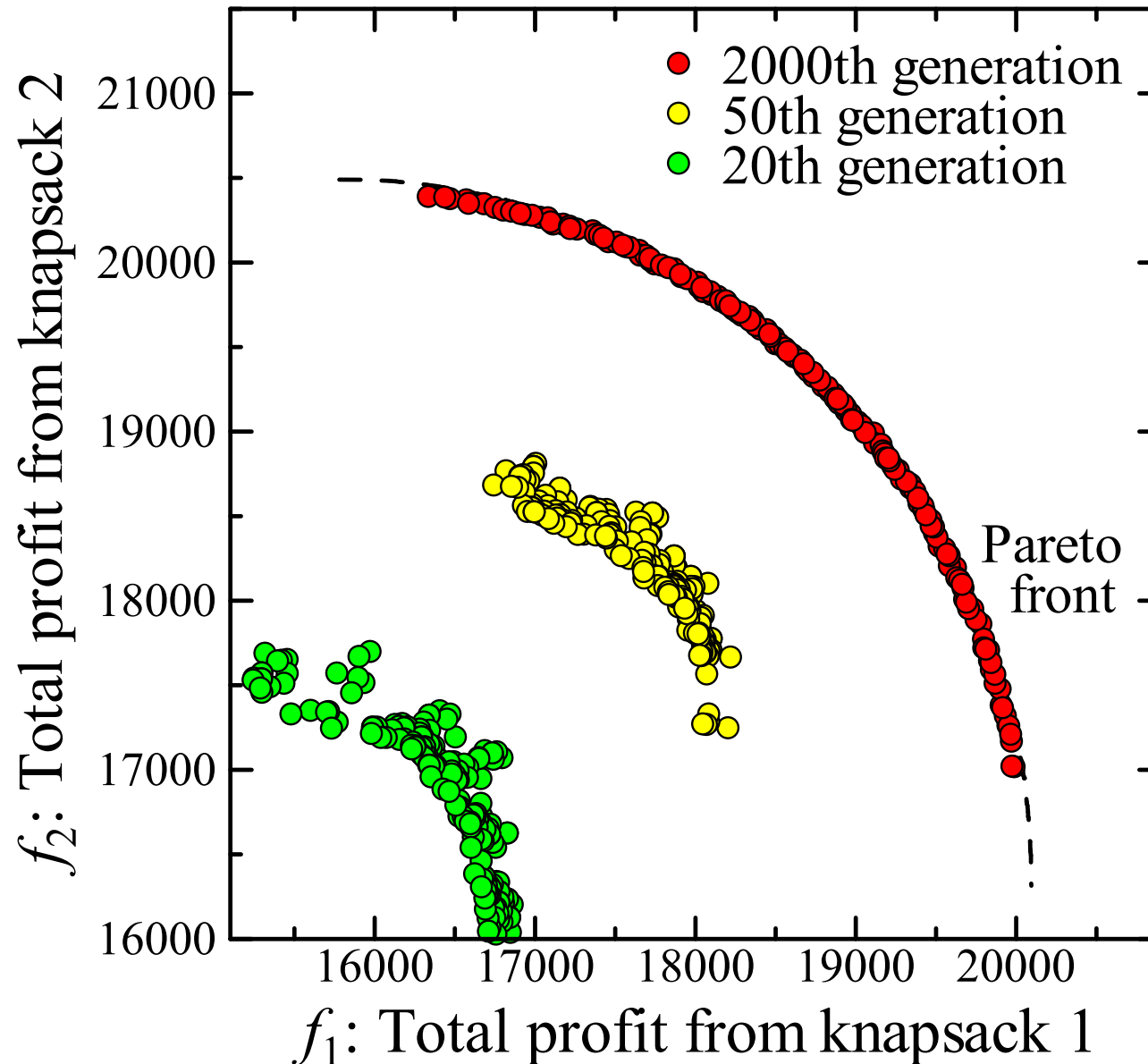
Surrogate-based Memetic Algorithm



- Local search is performed using the surrogate model.
- If a current solution is improved by local search, the improved solution is evaluated (i.e., exact fitness evaluation).
- All solutions in the main evolutionary part are evaluated exactly whereas local search is performed based on the surrogate model.

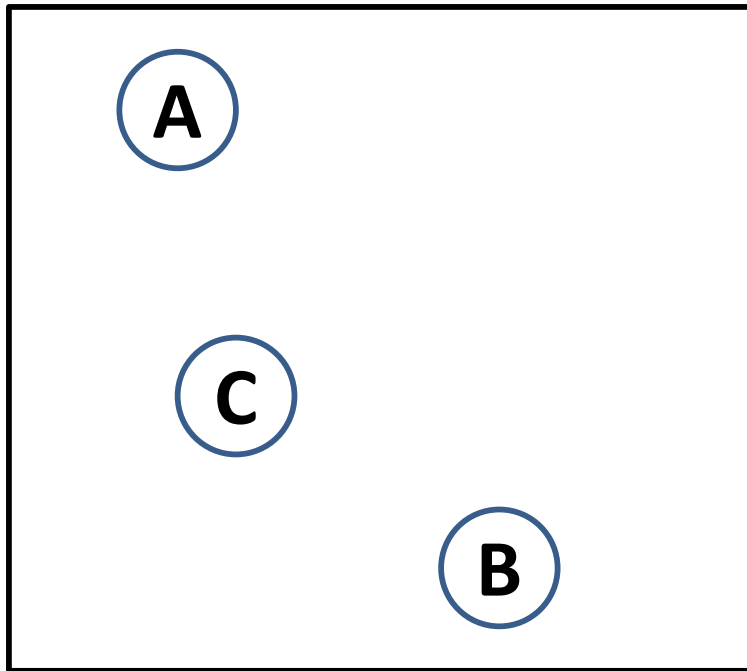
Example of Implementation:

Surrogate-based Multiobjective Algorithm

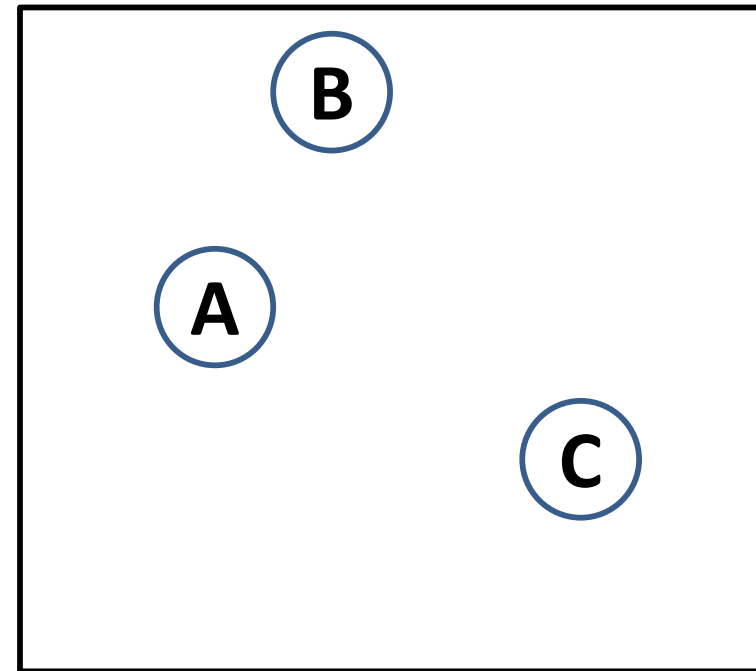


How to Use Surrogate Models in EMO Algorithms?

- (1) Function approximation for each objective function.
- (2) Function approximation for aggregated functions (e.g., scalarizing functions in MOEA/D).
- (3) Approximation of the Pareto dominance relation between two solutions $\mathbf{x}_i$ and $\mathbf{x}_j$: Non-dominated with each other, $\mathbf{x}_i$ is better, or $\mathbf{x}_j$ is better. (Special three-class classification problem)



Decision space

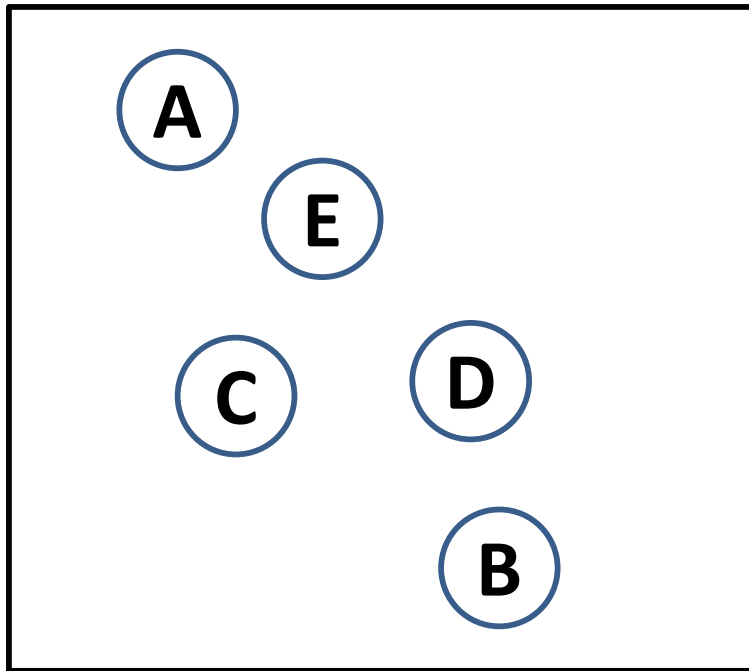


Objective space (minimization)

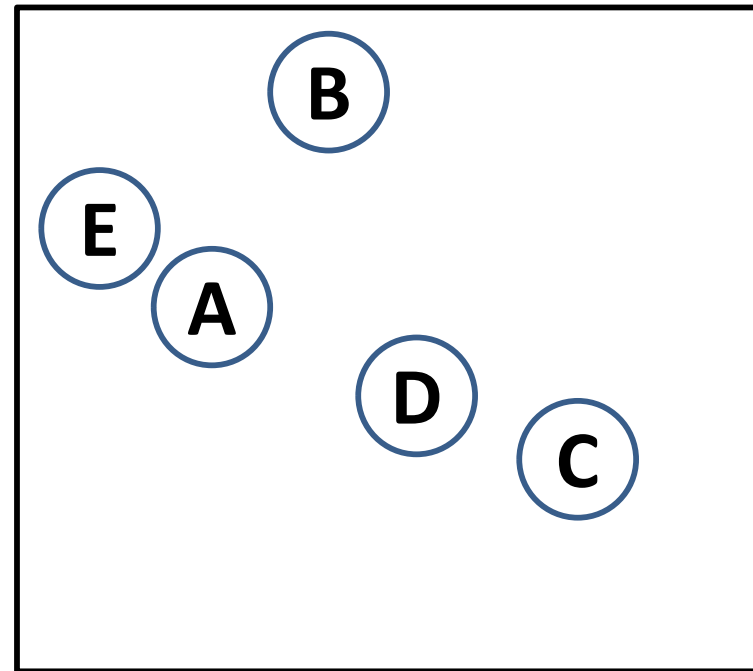
How to Use Surrogate Models in EMO Algorithms?

(4) Approximation of the comparison between two solutions $\mathbf{x}_i$ and $\mathbf{x}_j$ by a particular fitness evaluation mechanism in an EMO algorithm (e.g., NSGA-II): $\mathbf{x}_i$ is better or $\mathbf{x}_j$ is better.

(Special two-class classification problem)



Decision space

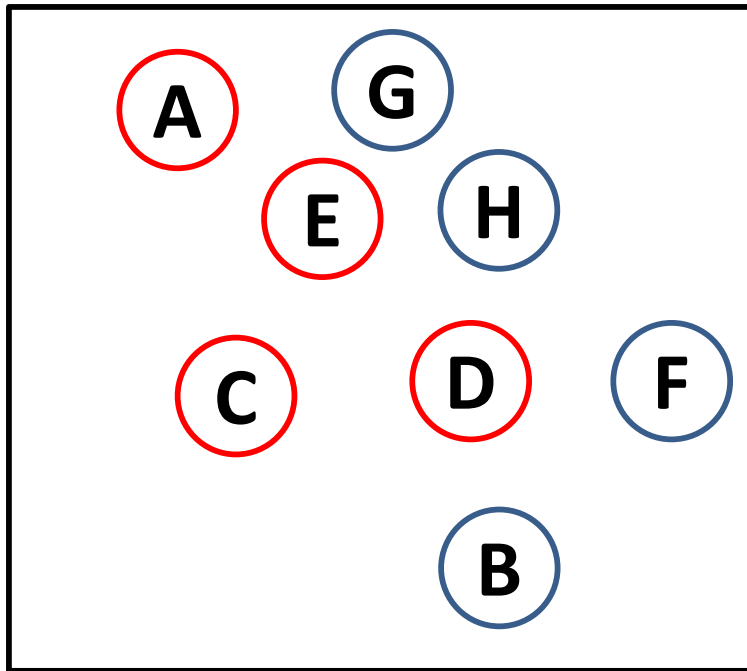


Objective space (minimization)

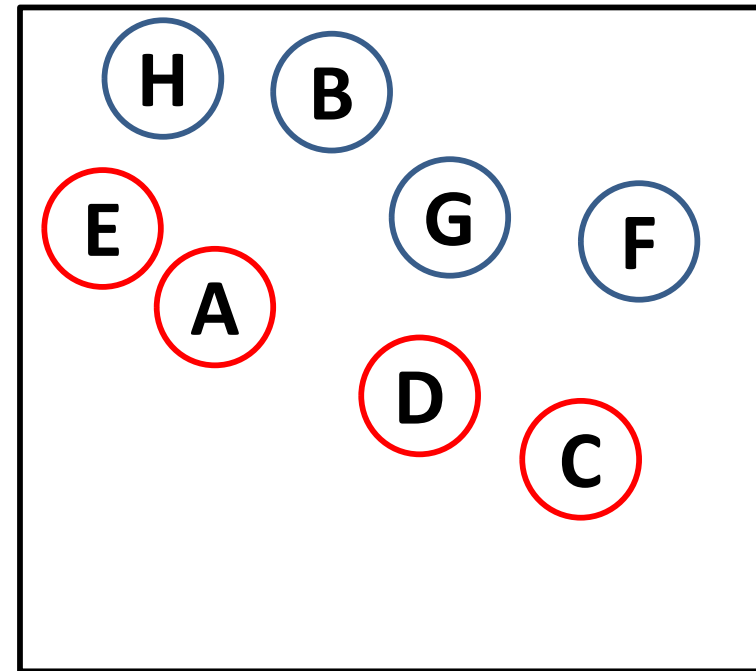
How to Use Surrogate Models in EMO Algorithms?

- (4) Approximation of the comparison between two solutions $\mathbf{x}_i$ and $\mathbf{x}_j$ by a particular fitness evaluation mechanism in an EMO algorithm (e.g., NSGA-II): $\mathbf{x}_i$ is better or $\mathbf{x}_j$ is better.
- (5) Classification of solutions in the current population between dominated and non-dominated solutions.

(Standard two-class classification problem)



Decision space



Objective space (minimization)

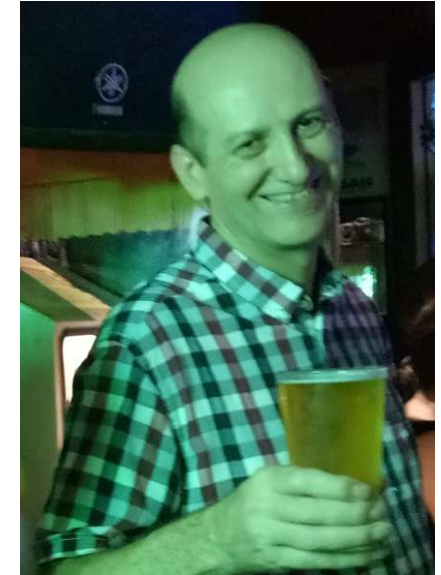
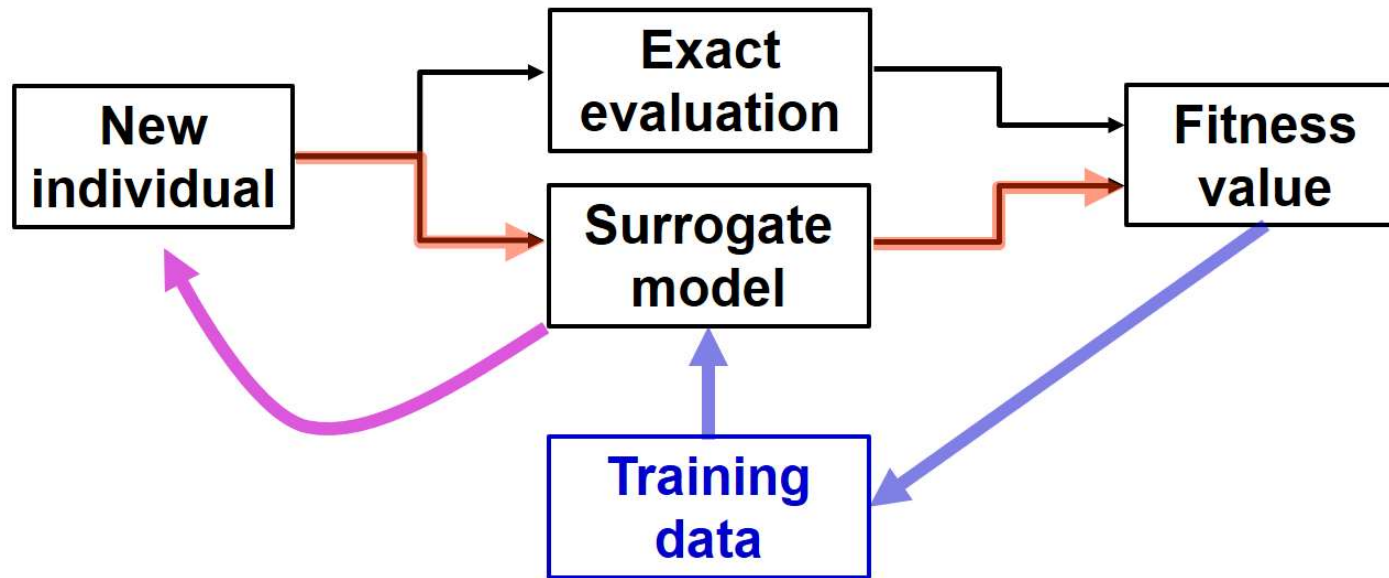
How to Use Surrogate Models in EMO Algorithms?

- (1) Function approximation for each objective function.
- (2) Function approximation for aggregated functions (e.g., scalarizing functions in MOEA/D).
- (3) Approximation of the Pareto dominance relation between two solutions $\mathbf{x}_i$ and $\mathbf{x}_j$: Non-dominated with each other, $\mathbf{x}_i$ is better, or $\mathbf{x}_j$ is better.
- (4) Approximation of the comparison between two solutions $\mathbf{x}_i$ and $\mathbf{x}_j$ by a particular fitness evaluation mechanism in an EMO algorithm (e.g., NSGA-II): $\mathbf{x}_i$ is better or $\mathbf{x}_j$ is better.
- (5) Classification of solutions in the current population between dominated and non-dominated solutions.

Q1. Which do you think a good strategy?

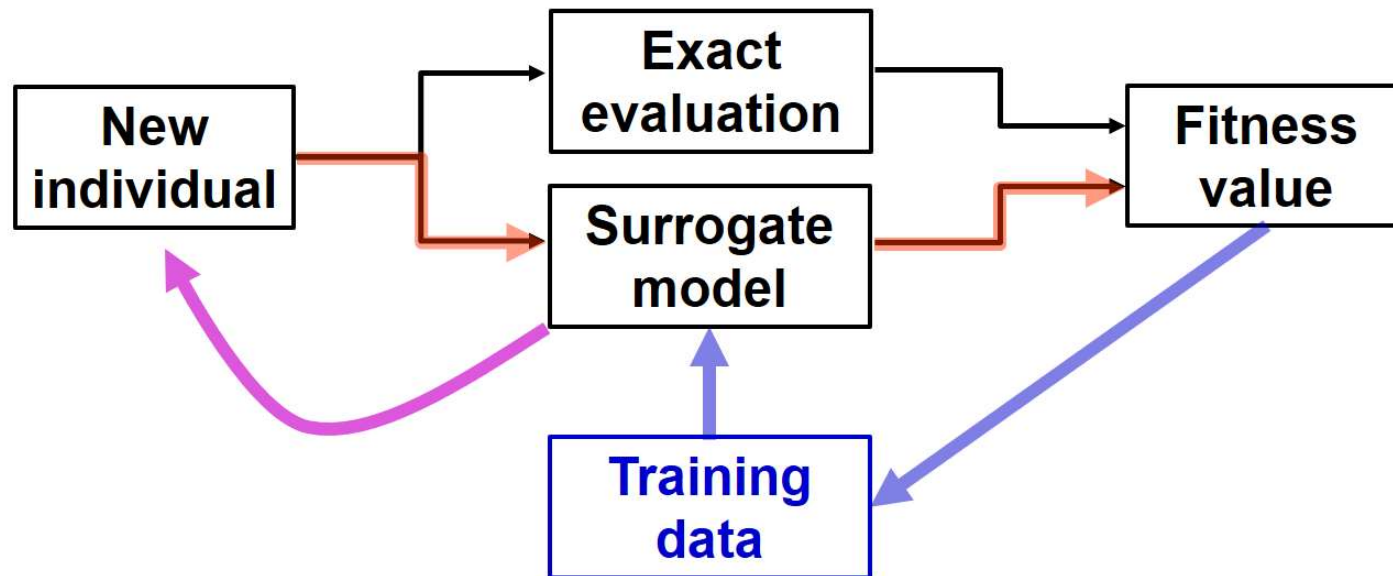
Example of Implementation:

Surrogate-based Interactive Algorithm



A surrogate model can be used to replace a human evaluation.

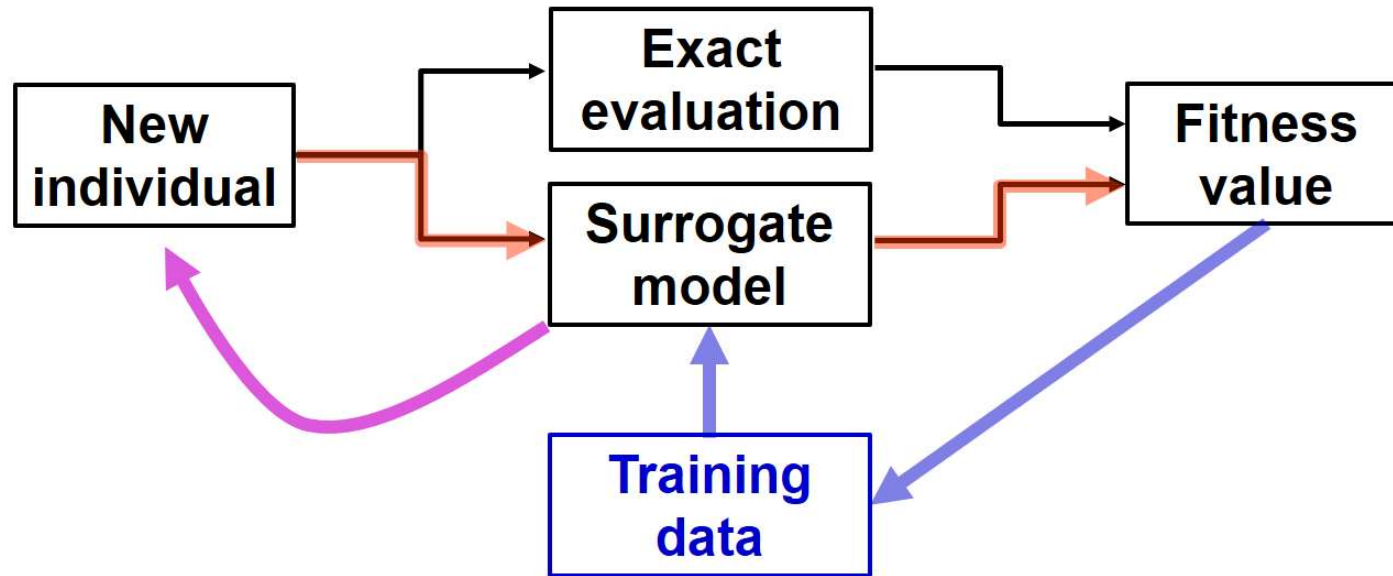
Other Related Topics to Surrogate Model



Model Complexity

An appropriate surrogate model may be different between an initial phase with a small training data set and a final phase with a large training data set. The requested accuracy for the surrogate model may be different between the initial phase and the final phase of search.

Other Related Topics to Surrogate Model



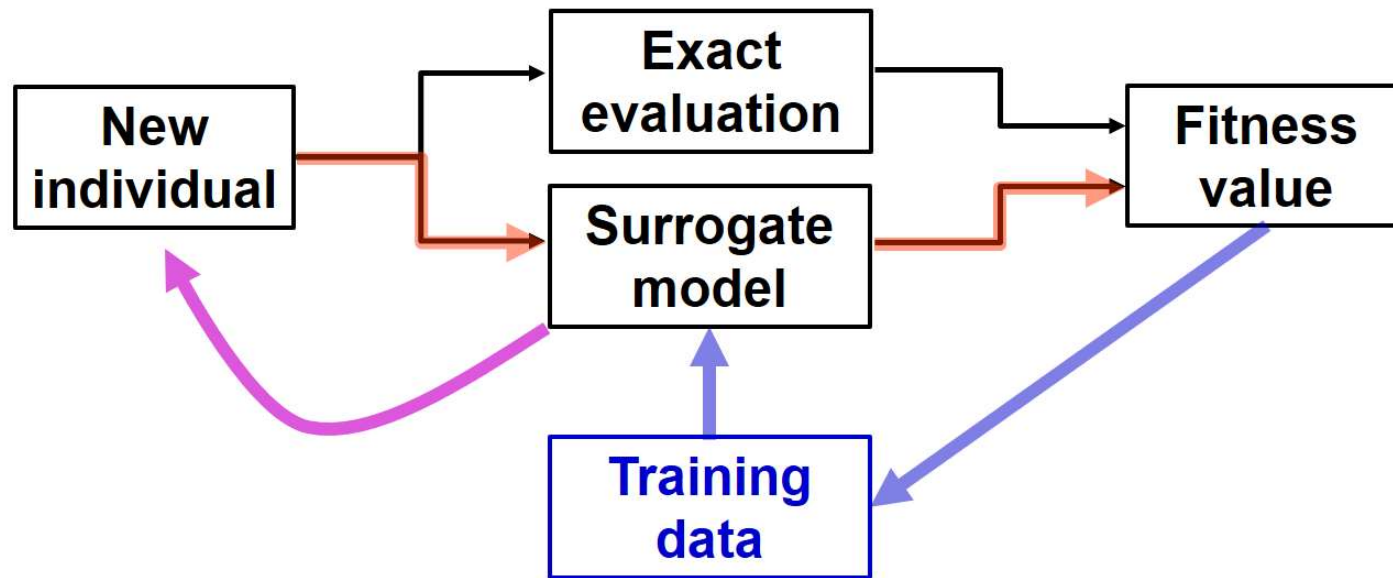
Semi-Supervised Learning

Through the execution of a surrogate-based algorithm, we have a small number of exactly evaluated solutions and a large number of solutions evaluated by the surrogate model.

Training Data: $\{ (\mathbf{x}_1, y_1), \dots, (\mathbf{x}_i, y_i), (\mathbf{x}_{i+1}, ?), \dots, (\mathbf{x}_{i+j}, ?) \}$ where i is small (e.g., 200) and j is large (e.g., 200,000).

Q. How can we utilize $\{(\mathbf{x}_{i+1}, ?), \dots, (\mathbf{x}_{i+j}, ?)\}$ in the learning of the surrogate model?

Other Related Topics to Surrogate Model



Active Learning

Training Data: $\{ (x_1, ?), (x_2, ?), \dots, (x_N, ?) \}$. The problem is to choose inputs for which the corresponding outputs should be calculated. (For a classification task, the problem is to choose patterns for which the corresponding classes should be assigned).